PROYECTO INTERAREAL

Proyecto elegido: Clínica Privada

Carpeta De Campo Individual

Alumno: Matías Basto

División Año: 5º3 T.M

Docentes: Nahuel Acosta - Gabriel Abello Tuinyla

INTEGRANTES DE GRUPO

Bautista Miyashiro

Luciano Villarroel

Santiago Cañetes

Martina Orlandi

Matías Basto

Escuela Técnica Nº32 DE 14 “Gral. José San Martín”

CABA, barrio Chacarita

Proyecto interdisciplinario

INDICE DE CARPETA DE CAMPO

PAGINA 3-4: Información sobre el proyecto y la utilidad de carpeta de campo

PAGINA 5: INFORMACION DE TODO LO REALIZADO EN JUNIO

PAGINA 5-18: INFORMACION DE TODO LO REALIZADO EN AGOSTO

PAGINA 19-37: INFORMACION DE TODO LO REALIZADO EN SEPTIEMBRE

PAGINA 37-58: INFORMACION DE TODO LO REALIZADO EN OCTUBRE

PAGINA 59-64: INFORMACION DE TODO LO REALIZADO EN NOVIEMBRE

Introducción:

El proyecto que nos tocó fue realizar un software para una Clínica Privada el cual busca ser funcional y útil. Este mismo proyecto se divide en el uso y la complementación de 5 materias; Base de datos, Análisis en Sistemas, Proyecto Informático 2, Redes y Programación Orientada a Objetos. Antes de empezar a realizar el proyecto, se nos otorgó un grupo de trabajo el cual fue aleatorio. El proyecto nos obliga a entender sobre programación, organizarnos, ser responsables, pensar y trabajar en grupo.

Objetivo de nuestro proyecto:

Lo que pensamos como objetivo de nuestro trabajo fue proveer una plataforma de gestión clínica centralizada en facilitar y garantizar la seguridad sobre la continuidad de la información asistencial, que se pueda reducirlos errores de administración médica. Con el desarrollo de este proyecto buscamos una plataforma que este orientada a la agilidad y el acceso inmediato, permitiendo que el medico visualice en un solo lugar y de manera rápida los antecedentes completos, alergias, evoluciones y diagnósticos previos del paciente que va atender. Esto elimina la pérdida de tiempo asociada a la búsqueda de fichas físicas en papel y evita tener que reiniciar la historia clínica en cada nueva consulta. También, el software pone un foco directo en la seguridad del paciente durante la prescripción de fármacos, previniendo errores en cualquier tipo de situaciones.

Objetivo de carpeta de Campo: La presente carpeta de campo tiene como objetivo registrar de manera detallada las actividades realizadas por Matías Basto en el marco de las materias: Programación Orientada a Objetos, Redes, Análisis de Sistemas, Base de Datos y Proyecto Informático II.

El propósito principal es llevar un control organizado de las tareas, tanto individuales como grupales, a fin de documentar el proceso de trabajo, los avances y las dificultades encontradas.

Los registros incluirán la siguiente información básica en cada entrada: fecha, lugar donde fue realizada la actividad, así como una descripción objetiva de lo que se hizo. Además, se añadirán observaciones o comentarios que permitan reflexionar sobre el desarrollo del trabajo.

Este documento servirá como evidencia del compromiso y participación de Matías Basto, y permitirá evaluar de forma clara el progreso alcanzado durante el período establecido.

Inicio De Proyecto:

24/06/26

Lugar: Aula Normal 5° 3

Hora 9:40

Actividad:

Hoy fueron sorteados los grupos del proyecto de forma aleatoria. Una vez nos enteramos de quienes nuestros compañeros con los que vamos hacer el proyecto, los profesores nos pidieron que nos sentáramos para poder terminar de explicar cómo va ser el proyecto, los requisitos y mostrarnos las distintas opciones de proyectos que podíamos elegir. Una vez terminaron de hablar y explicar todo, nos pudimos juntar con el grupo para elegir el proyecto el cual íbamos a desarrollar. Al final terminamos eligiendo la Clínica Privada porque se veía interesante.

SIN ACTIVIDAD RELEVANTE

06/08/26

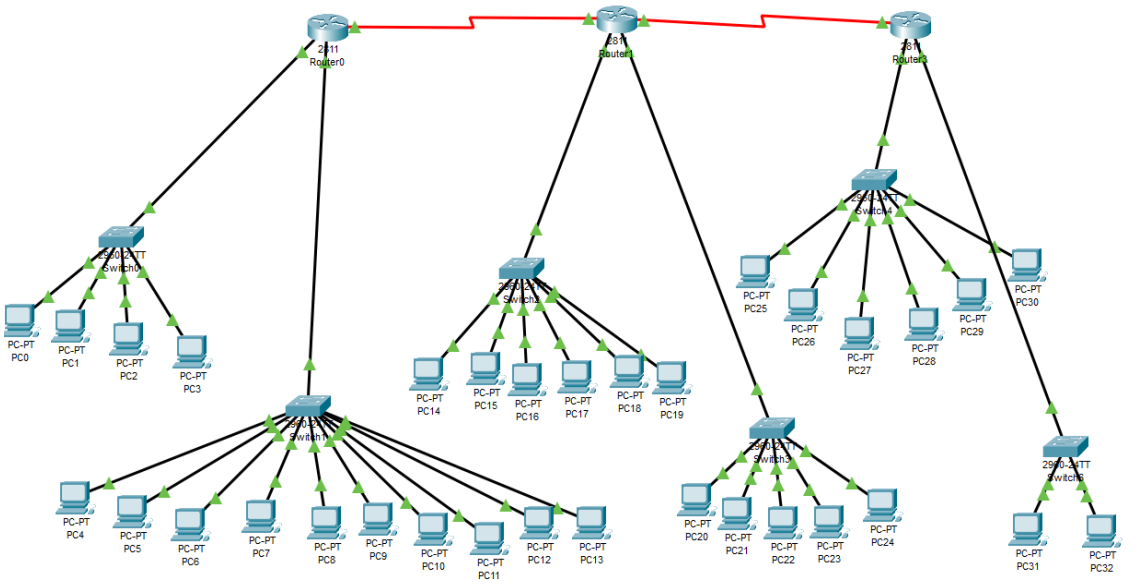
Lugar: Aula Informática

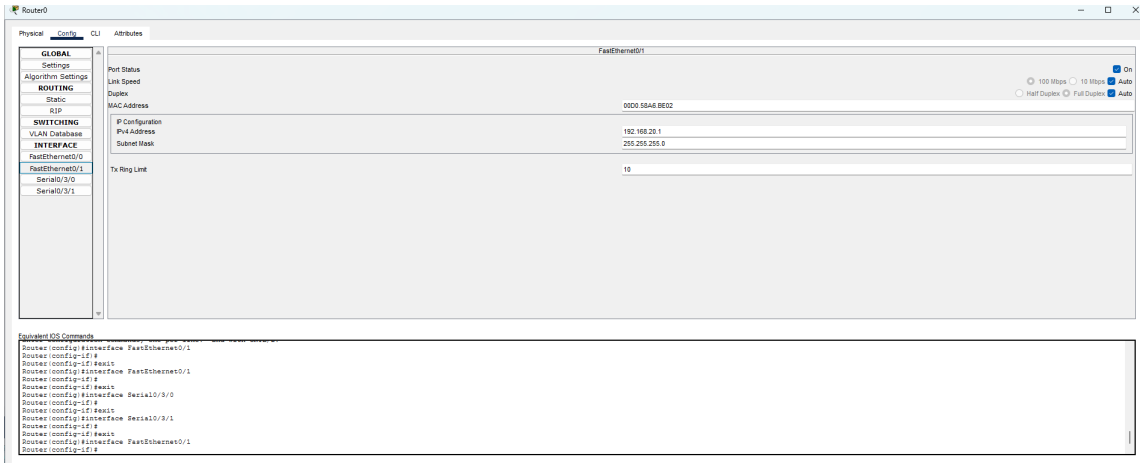
Hora 10:00

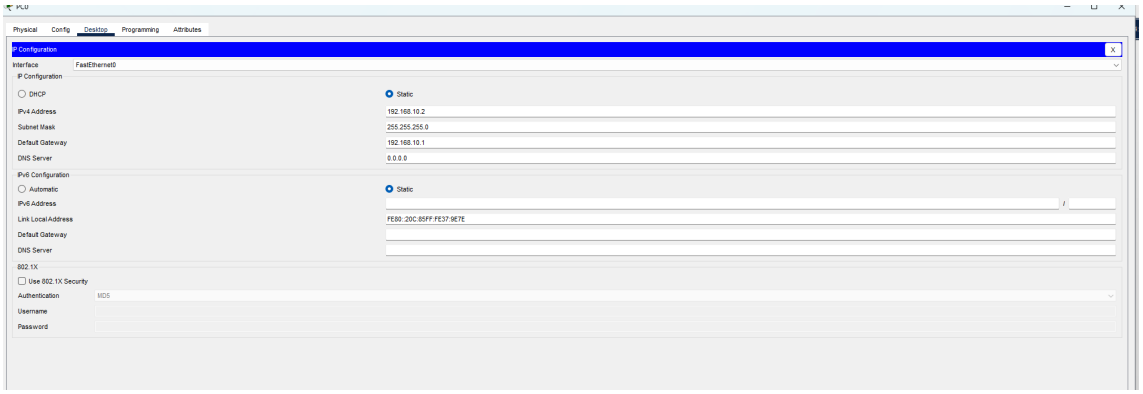
Actividad:

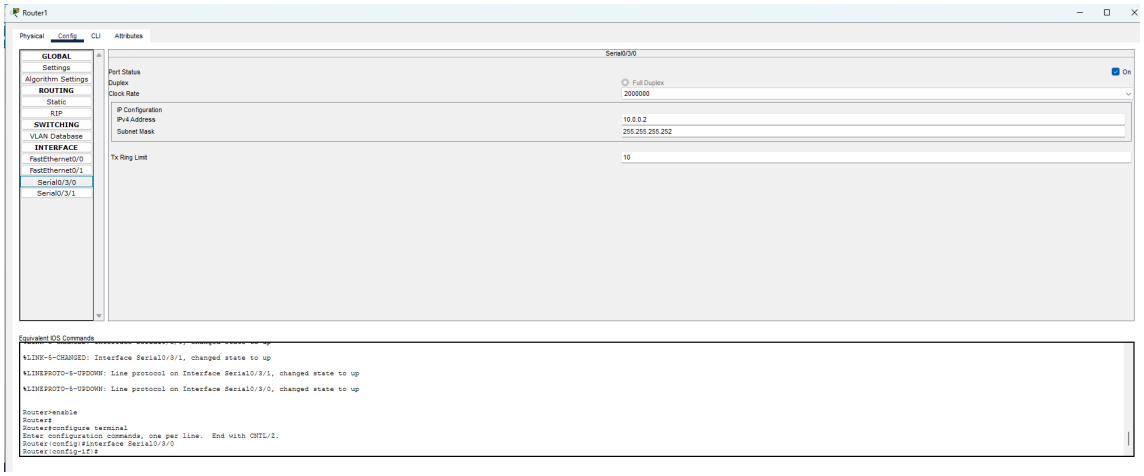
El día de hoy oficialmente pudimos establecer cuando íbamos a hacer la entrevista para continuar el proyecto, la fecha pactada fue para el día sábado 08/06/26, gracias a la ayuda de cañetes que consiguió a una persona que trabajaba en el ámbito de salud. Una vez habíamos planteado una idea de las preguntas para la entrevista, proseguimos a realizar la actividad de Redes para el proyecto, ya que nuestro Profesor Nahuel, nos pasó una infraestructura para que empecemos con el presupuesto inicial del proyecto y una beta del armado de la red. Con mis compañeros Bautista Miyashiro y Santiago cañetes, planteamos una primera idea de cómo podríamos hacer la red. Una vez planteada beta del armado, yo me encargué de realizar toda la configuración y conexión de la red. Primero configure los routers asignándoles su ip, y configuración la ip de la

conexión entre router, como eran 3 los conectamos por un Serial. Una vez configure los routers, conectamos las computadoras a los switchs, asignándoles la ip e hicimos pings para probar que anduviera todo bien. Lo probamos e iba bien, llamamos al profesor Nahuel para que nos diga como lo veía, lo cual salió todo bien, nos terminó aprobando la primera red. Antes de terminar el día, nos juntamos para asignar las tareas de Análisis. A mí me toco como primera instancia pensar y realizar el Diagrama de Gantt.









07/08/26

Sin actividades realizadas este día

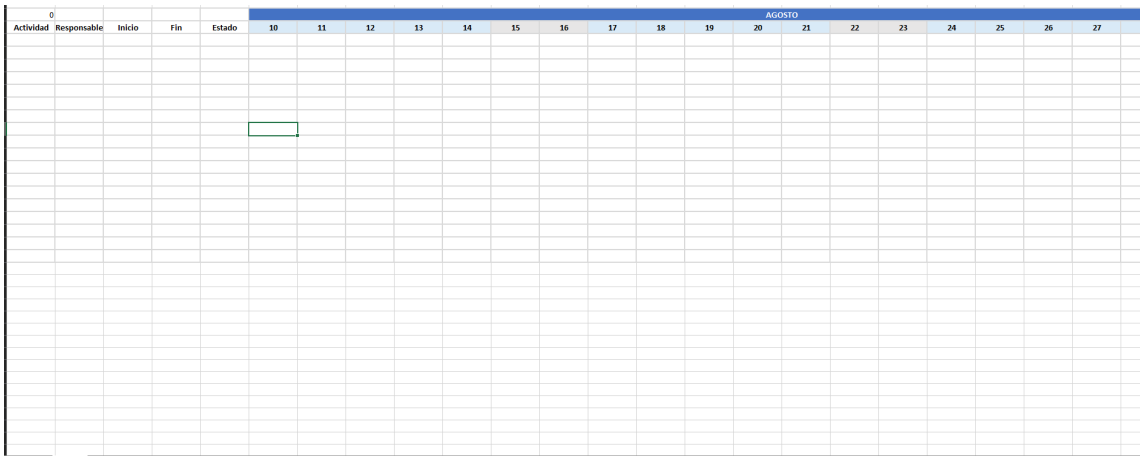
10/08/26

Lugar: Aula Informática

Hora: 10:00

Actividad:

Hoy pudimos avanzar más con el proyecto, ya que Santiago Cañetes pudo hacer la entrevista y pasarla a limpio. Así que, me puse a pensar una idea de cómo realizar el diagrama de Gantt. Creé un Excel y más o menos configuré para poder realizar y plasmar las ideas del Gantt. Antes que terminara el día, pude terminar más o menos lo visual del diagrama con ayuda de Miyashiro, aunque todavía me faltaba plasmar las actividades.



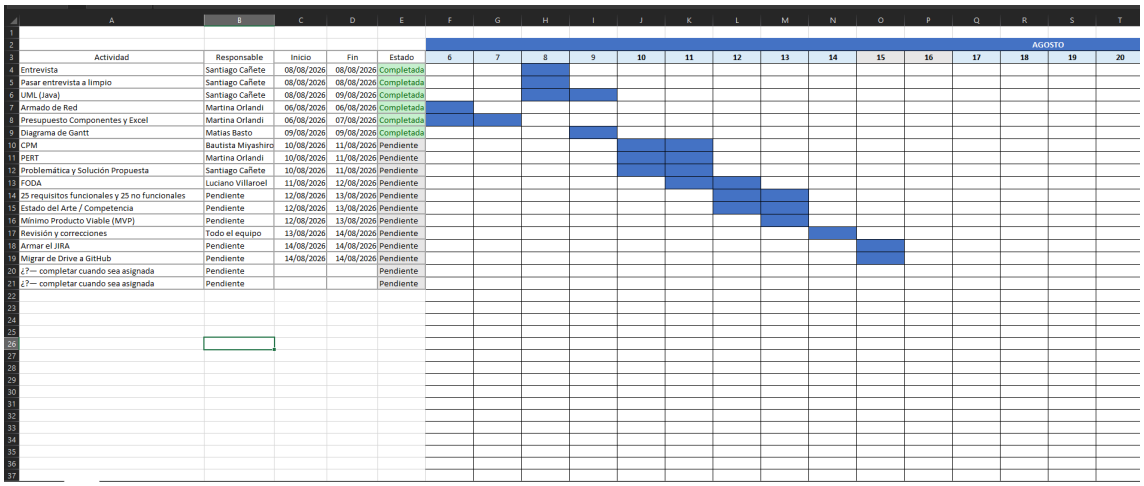
11/08/26

Lugar: Aula Informática

Hora: 9:55

Actividad:

Hoy seguí realizando el diagrama de Gantt. Como no había prestado mucha atención a la explicación del diagrama al profesor Gabriel me toco ver un video de cómo hacerlo (https://www.youtube.com/watch?v=cqhMP9rlbG0). Una vez que vi el video, seguí realizando el diagrama. Plasme las actividades, responsables, fechas y el estado. Además, se me fue asignada una nueva tarea, Estado del Arte / Competencia, la cual no sabía que era así que decidí .



Actividad	Responsable	Inicio	Fin	Estado
Entrevista	Santiago Cañete	08/08/2026	08/08/2026	Completada
Pasar entrevista a limpio	Santiago Cañete	08/08/2026	08/08/2026	Completada
UML (Java)	Santiago Cañete	08/08/2026	09/08/2026	Completada
Armado de Red	Martina Orlandi	06/08/2026	06/08/2026	Completada
Presupuesto Componentes y Excel	Martina Orlandi	06/08/2026	07/08/2026	Completada
Diagrama de Gantt	Matias Basto	09/08/2026	09/08/2026	Completada
CPM	Bautista Miyashiro	10/08/2026	11/08/2026	Pendiente
PERT	Martina Orlandi	10/08/2026	11/08/2026	Pendiente
Problemática y Solución Propuesta	Santiago Cañete	10/08/2026	11/08/2026	Pendiente
FODA	Luciano Villaroel	11/08/2026	12/08/2026	Pendiente
25 requisitos funcionales y 25 no funcionales	Pendiente	12/08/2026	13/08/2026	Pendiente
Estado del Arte / Competencia	Pendiente	12/08/2026	13/08/2026	Pendiente
Mínimo Producto Viable (MVP)	Pendiente	12/08/2026	13/08/2026	Pendiente
Revisión y correcciones	Todo el equipo	13/08/2026	14/08/2026	Pendiente
Armar el JIRA	Pendiente	14/08/2026	14/08/2026	Pendiente
Migrar de Drive a GitHub	Pendiente	14/08/2026	14/08/2026	Pendiente
¿?— completar cuando sea asignada	Pendiente			Pendiente
¿?— completar cuando sea asignada	Pendiente			Pendiente

AGOSTO: 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20

12/08/26

Lugar: Laboratorio

Hora: 7:40

Actividad:

No se realizó actividades el día de hoy

13/08/26

Lugar: Laboratorio

Hora: 9:55

Actividad:

Hoy pude terminar mi primera idea como Gantt, modifique las posiciones nuevamente de las actividades, las separe por categoría y agregue algunas más. Además, me puse a hacer el Estado del Arte / Competencia, investigué primero una idea como plasmarlo en un Word (https://www.youtube.com/watch?v=wooqf2GvEg0)



Actividad	Responsable	Inicio	Fin	Estado
ANÁLISIS				
Entrevista	Santiago Cañete	8/8/2026	8/8/2026	Completada
Pasar entrevista a limpio	Santiago Cañete	8/8/2026	8/8/2026	Completada
Diagrama de Gantt	Matias Bastia	8/8/2026	9/8/2026	Completada
CPM	Bautista Miyashiro	10/8/2026	11/8/2026	Completada
PERT	Martina Orlandi	10/8/2026	11/8/2026	Pendiente
FODA	Luciano Villarroel	10/8/2026	11/8/2026	Completada
25 requisitos funcionales y 25 no funcionales	Luciano Villarroel	11/8/2026	12/8/2026	Completada
Problemática y Solución Propuesta	Santiago Cañete	12/8/2026	13/8/2026	Completada
Estado del Arte / Competencia	Matias Bastia	12/8/2026	13/8/2026	Completada
MVP	Santiago Cañete	13/8/2026	14/8/2026	Completada
Análisis Tecnológico	Luciano Villarroel	12/8/2026	13/8/2026	Completada
Inicio de los Diagramas de flujo y de contexto	Luciano Villarroel	12/8/2026	13/8/2026	Completada
Armar el JIRA	Santiago Cañete	13/8/2026	13/8/2026	Completada
Migrar de Drive a GitHub	Luciano Villarroel	14/8/2026	14/8/2026	Completada
BASE DE DATOS				
PENSAR EL DER	Matias Bastia	15/08/2026	15/8/2026	Completada
DER BETA	Matias Bastia	16/8/2026	16/8/2026	Completada
Diccionario de datos	Matias Bastia	17/8/2026	17/8/2026	Completada
Beta de la base de datos	Matias Bastia	18/8/2026	20/8/2026	Completada
PENSAR PROCEDIMIENTOS	Matias Bastia	20/8/2026	22/8/2026	Completada
PENSAR VISTAS	Matias Bastia	20/8/2026	23/8/2026	Completada
PROYECTO INFORMÁTICO				
Boceto de las páginas web	Bautista Miyashiro	15/8/2026	18/8/2026	Completada
Funcionalidades	Bautista Miyashiro	18/8/2026	21/8/2026	Completada
REDES				
Primer Presupuesto	Martina Orlandi	8/8/2026	9/8/2026	Completada
Beta Armado de RED	Martina Orlandi	8/8/2026	8/8/2026	Completada
Configuración de DHCP y DNS BETA	Martina Orlandi	29/8/2026	30/8/2026	Completada
Display sección 1 hasta la 10 inclusive en CPT	Martina Orlandi	22/8/2026	30/8/2026	Completada
Presupuesto FINAL	Martina Orlandi	24/8/2026	30/8/2026	Completada
Configuración de servidor DHCP y DNS final	Martina Orlandi	29/8/2026	31/8/2026	Completada
POO				
pensar como realizar el uml	Santiago Cañete	15/8/2026	16/8/2026	Completada
pensar funciones	Santiago Cañete	17/8/2026	18/8/2026	Completada
pensar clases	Santiago Cañete	18/8/2026	19/8/2026	Completada
pensar uso de main	Santiago Cañete	19/8/2026	20/8/2026	Completada
Pensar uso de CRUD : PLANTEAR EN UN DOCX	Santiago Cañete	20/8/2026	21/8/2026	Completada
Diagrama UML finalizado :	Santiago Cañete	21/8/2026	23/8/2026	En proceso
CRUD sin interfaz gráfica :	Santiago Cañete	23/8/2026	26/8/2026	Completada
CLASES beta1	Santiago Cañete	26/8/2026	28/8/2026	Completada
MAIN beta1	Santiago Cañete	28/8/2026	31/8/2026	Completada

14/08/26

Lugar: Laboratorio

Hora: 9:55

Actividad:

Hoy realicé de 0 el Estado del Arte / Competencia, después de haberme visto el video del día de ayer, empecé a buscar información mediante por google sobre lo más avanzado que se ha alcanzado sobre un tema y problema especifica. Después de investigar, me puse a plasmar la idea de estado del arte / competencia de nuestro proyecto, finalizando esto, le pregunte al líder como veía el estado del arte, lo cual me dio luz verde para que lo suba al drive y después al github de nuestro proyecto.

Estado del Arte / Competencia

Somos estudiantes de la escuela técnica n32, estamos llevando a cabo un software de una clínica privada. El proyecto que abordamos es un problema crítico que se nos presente en el ámbito de la salud, que es la discontinuidad de la información asistencial, el margen de error humano en la administración de fármacos en situaciones de urgencia y la falta de unificación de criterios médicos en áreas de terapia e internación

Estado del Arte

Hoy en día, el estado del arte nos muestra un gran avance y una tendencia a la digitalización de las cosas y la centralización de los historiales clínicos de los pacientes. Los hospitales van poco a poco pasando de lo antiguo físico, o sea, las carpetas físicas clínicas a algo más digital, pasando a los historiales clínicos electrónicos, los cuales permiten a los médicos el acceso inmediato y seguro a los datos desde cualquier sala de atención. Igualmente, como también se busca facilitar las cosas, también hay que ser precisos en la transcripción de una consulta, registrando la fecha, hora, los síntomas del paciente, la evolución del paciente a lo largo de sus consultas, los medicamentos que se le administra en cada consulta y los involucrados en atenderlo tanto en lo administrativo (entregar el turno), la consulta y quien le entrega los medicamentos. De esa manera, se evitan problemas de entendimiento entre médicos y no pasa lo que suele pasar, ir a atenderte con alguien un día, al otro día atenderte con uno nuevo y tener que empezar de 0. En este caso, debe ser necesario contar con herramientas de consulta rápida capaces de consolidar en una sola vista los antecedentes del paciente, sea cual sea la consulta por la que llegue y medicamentos que se le hayan recetado.

Análisis de la Competencia

En los hospitales de hoy en Argentina, existen 3 alternativas diferentes con las que se encuentra nuestro trabajo. Por un lado, el mítico registro en papel (Las carpetas físicas), el cual convive en muchos hospitales públicos todavía, como el Argerich, Hospital Santojanni, Hospital Ramos Mejía, etc. Es un método el cual no es malo, pero presenta errores que hace que

pegue en el palo y no sea conveniente usarlo hoy en día. Como, por ejemplo; el riesgo de extravío, la ilegibilidad de las anotaciones, la lentitud para localizar antecedentes en situaciones complejas y la pérdida de información durante los traspasos de turno. Bien en estos hospitales las consultas ambulatorias o consultorios externos se cargan en el sistema digital, cuando un paciente ingresa por la guardia o pasa a quirófano, se le arma una carpeta física de cartón y mantiene esos riesgos que mencioné antes. La segunda alternativa son los sistemas informáticos viejos de administración. Las clínicas y sanatorios privados de Argentina que sí tienen computadoras suelen usar software de gestión médico de administración, los cuales no están mal, pero se centran en el diseño orientado principalmente de la facturación, la asignación de turnos y cargar bonos, pero dejando de lado el lado médico, al momento de entrar a la historial clínico, es un sistema duro, lento y muy tedioso. Por última alternativa, se usa los chats de whatsapp lo cual es algo informal a nivel profesional porque es algo donde la información puede perderse y por un error ser vulnerada por alguien externo, además de que carece validez en el ámbito legal ya que esa información nunca es pasada a ninguna ficha del paciente en la ficha del paciente y no garantiza el debido registro oficial de las decisiones tomadas.

Soluciones

Comparando los distintos sistemas que vemos en los hospitales cotidianamente, nuestra propuesta quiere llevar las cosas a la facilidad para los médicos y seguridad de los pacientes. Mientras que los hospitales tienen un acceso lento y dependiente de que no se pierda nada de su PROPIA información, la propuesta trata de facilitar las consultas y concentrar la información desde un ámbito digital (o sea, una base de datos o por el estilo). Por otro lado, las opciones tradicionales sufren mucho por la perfección para mantener una organización y que no se pierda ningún documento. La propuesta registra cada atención médica que tenga cada paciente con la fecha, la medicación recomendada, síntomas, evolución y médicos responsables. Además, terminar con las letras feas de los médicos, resaltar los antecedentes de un paciente y los medicamentos recetados.

Ventaja Competitiva

La ventaja competitiva que tenemos con nuestro proyecto es la agilidad y centrado en la consulta rápida. Este software se consolida en un solo panel donde la información del paciente está segura y sea fácil de visibilizar. Esta optimización reduce de manera significativa los tiempos de lectura previa a la consulta, reduce la probabilidad de cometer errores por falta de antecedentes e intenta promover una atención médica mucho más rápida, eficaz y coordinada

17/08/26

Lugar: Laboratorio

Hora: 9:55

Actividad:

No se realizaron actividades el día de hoy

18/08/26

Lugar: Laboratorio

Hora: 9:55

Actividad:

No se realizaron actividades el día de hoy

19/08/26

Lugar: Aula Informática

Hora: 7:40

Actividad:

No se realizaron actividades el día de hoy

20/08/26

Lugar: Laboratorio

Hora: 9:55

Actividad:

No se realizaron actividades el día de hoy

21/08/26

Lugar: Laboratorio

Hora: 9:55

Actividad:

No se realizaron actividades el día de hoy

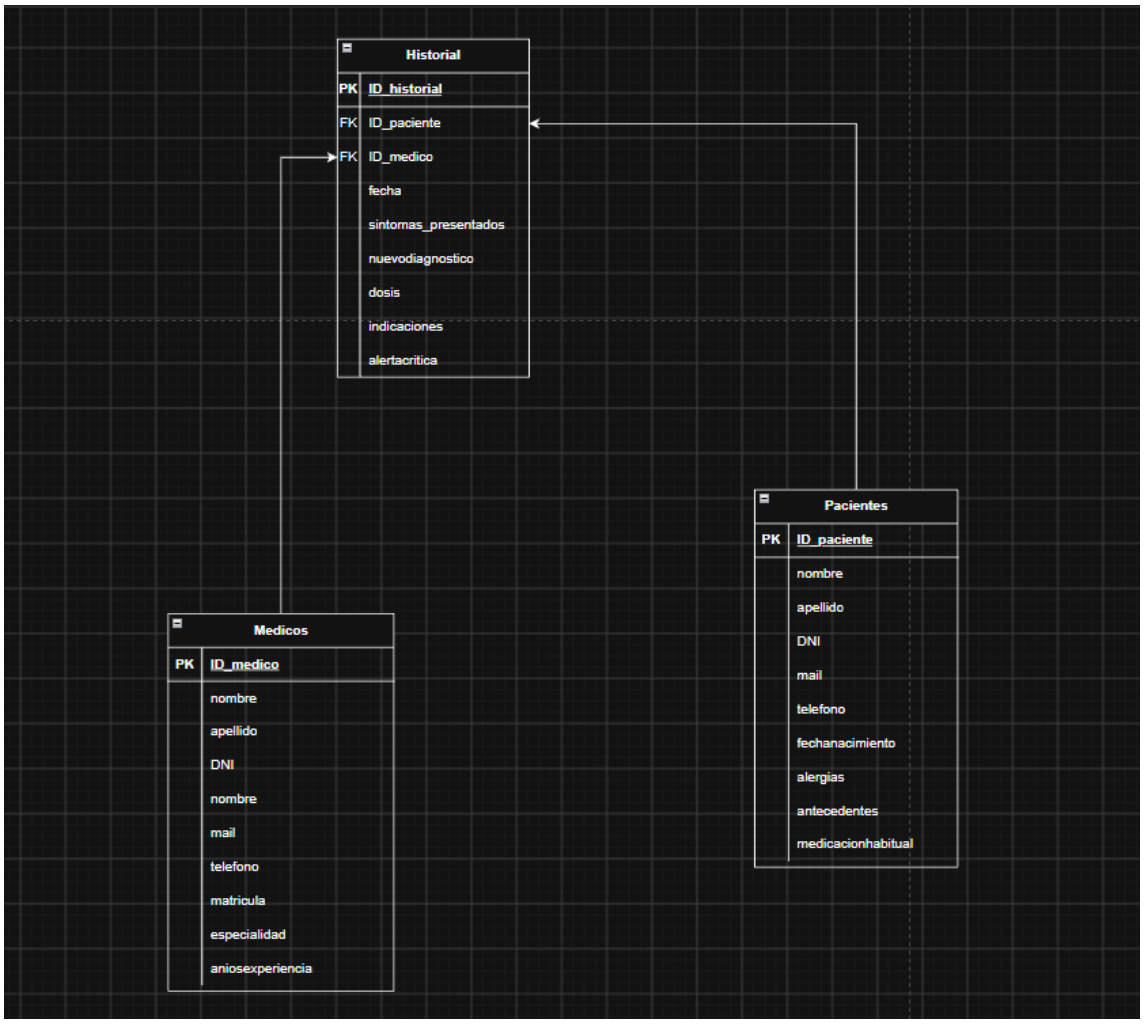
24/08/26

Lugar: Aula Informática

Hora: 9:55

Actividad:

Hoy fui asignado para encargarme la parte de Base de Datos del proyecto. Empecé viendo el UML y MVP para darme una idea de cómo podría encarar el código, pensando en las tablas que íbamos a usar y empecé haciendo una idea del DER básica para tener una base para empezar con la primera idea del código. Pensé en las tablas médico, paciente, historial



25/08/26

Lugar: Laboratorio

Hora: 9:55

Actividad:

No se realizaron actividades el día de hoy

26/08/26

Lugar: Aula informática

Hora: 7:40

Actividad:

No se realizaron actividades el día de hoy

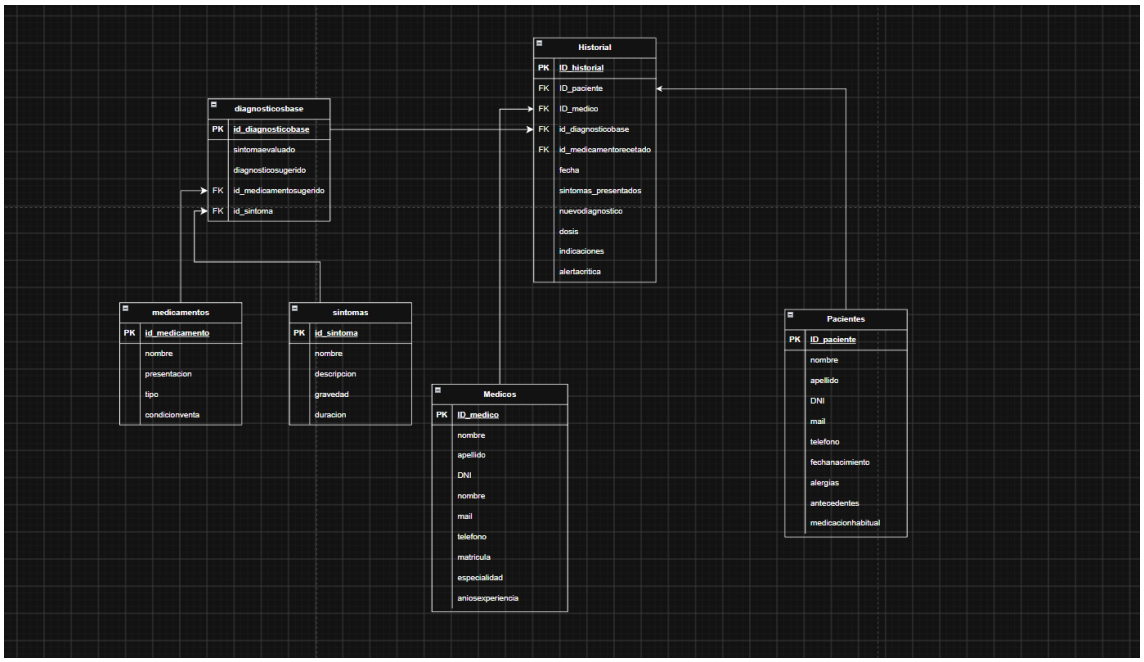
27/08/26

Lugar: Laboratorio

Hora: 9:55

Actividad:

Hoy seguí modificando el DER, agregué nuevas tablas y modifiqué algunos atributos que no me convencían. Puse unas nuevas tablas como medicamento y diagnostico base, lo agregué para que el sistema pueda tener una base de diagnósticos sugeridos a partir de los síntomas que presenta un paciente. O sea, el médico primero realiza la revisión del paciente y obtiene los datos y síntomas. Luego, carga esos datos en el sistema. De ahí, la maquina utiliza la información almacenada en diagnostico base para buscar una coincidencia y mostrar un posible diagnóstico como sugerencia. Para terminar, me puse a pensar en posibles vistas y procedimientos iniciales para el programa.



28/08/26

Lugar: Laboratorio

Hora: 9:55

Actividad:

Como termine la primera idea de la base de datos, plasme lo hecho anteriormente en el DER a una base de datos para su respectiva entrega, poniendo todos los atributos que ya habíamos pensando con anterioridad.

```
drop database if exists clinica_privada;
create database clinica_privada;
use clinica_privada;
create table pacientes (
    id_paciente int primary key auto_increment,
    dni varchar(20),
    nombre varchar(50),
    apellido varchar(50),
    fechanacimiento date,
    alergias varchar(50),
    antecedentes varchar(50),
    medicacionhabitual varchar(50)
);

create table medicos (
    id_medico int primary key auto_increment,
    dni varchar(20),
    nombre varchar(50),
    apellido varchar(50),
    matricula varchar(20),
    especialidad varchar(50),
    aniosexperiencia int
);

create table medicamentos (
    id_medicamento int primary key auto_increment,
    nombrecomercial varchar(100),
    presentacion varchar(100),
    tipo varchar(50),
    condicionventa varchar(50)
);
```

```sql
create table sintomas (
    id_sintoma int primary key auto_increment,
    nombre varchar(100),
    descripcion varchar(100),
    gravedad varchar(20),
    durabilidad varchar(50)
);

create table diagnosticosbase (
    id_diagnosticobase int primary key auto_increment,
    sintomaevaluado varchar(100),
    diagnosticosugerido varchar(150),
    id_medicamentosugerido int,
    foreign key (id_medicamentosugerido) references medicamentos(id_medicamento)
);

create table historial (
    id_historial int primary key auto_increment,
    id_paciente int,
    id_medico int,
    fecha datetime,
    sintomas_presentados varchar(150),
    id_diagnosticobase int,
    id_medicamentorecetado int,
    nuevodiagnostico varchar(150),
    dosis varchar(150),
    indicaciones varchar(150),
    alertacritica varchar(100),
    foreign key (id_paciente) references pacientes(id_paciente),
    foreign key (id_medico) references medicos(id_medico),
    foreign key (id_diagnosticobase) references diagnosticosbase(id_diagnosticobase),
    foreign key (id_medicamentorecetado) references medicamentos(id_medicamento)
);
```

31/08/26

Lugar: Laboratorio

Hora: 9:55

Actividad:

Hoy fue el día de la primera evaluación de recorrido del proyecto, lamentablemente los profesores no nos llegaron a llamar, así que nos dijeron que el miércoles seguirán llamando a los grupos. Antes de acabar la clase me puse a pensar algunos procedimientos y vistas que podríamos hacer para nuestra clínica



1/09/26

Lugar: Laboratorio

Hora: 9:55

Actividad:

No se realizaron actividades el día de hoy

2/09/26

Lugar: Aula informática

Hora: 7:40

Actividad:

El día de hoy llegó el 2do día de revisión de proyecto, apenas llegamos los profesores nos pidieron que organicemos todo en las computadoras antes que llegue nuestro respectivo turno para la revisión. Nos llamaron y empezaron a revisar nuestro proyecto. Dentro de un rato revisando, nos dijeron muchas cosas; Nos dijeron que no estábamos siguiendo el camino que ellos estaban esperando, que estábamos tomando un camino distinto, que el MVP está incompleto y que el camino de división de tareas no es la mejor de todas. Nos recomendaron plantear de 0 todo lo que es la división de tareas y rehacer el MVP con claridad sus consignas. Además, en mi base de datos me dijeron que la podría modificar, que necesitaban una tabla de especialidades, en vez que sea un atributo, además, modificar algunas tablas que no tenían mucho sentido. Terminamos la entrevista y con el grupo hablamos sobre la distribución de tarea, decidimos terminar las actividades de agosto que nos faltaban como veníamos y a partir de septiembre empezar con una nueva distribución. Así que, me puse con el líder a plasmar las actividades que nos quedaron pendientes en la revisión así las hacemos antes de empezar devuelta en septiembre

PENDIENTES PARA SEGUIR AGOSTO				
POO				
Realizar el MVP Definitivo para Java.	Santiago Cañete			Pendiente
Añadir la Clase Enfermedad y sus clases Hijas al codigo de java.	Martina Orlandi	6/9/2026	6/9/2026	Pendiente
Añadir las Funciones de Enfermedades Al crud. (Agregar,editar,Mo	Martina Orlandi			Pendiente
Añadir Enfermedades al UML y Actualizarlo	Luciano Villarroel			Pendiente
Añadir la clase Turno y sus clases Hijas Al codigo de java	Luciano Villarroel			Pendiente
Añadir las Funciones de Turno Al crud. (Agregar,editar,Mostrar,Elin	Matias Basto			Pendiente
Añadir Turno al UML y actualizarlo.	Matias Basto			Pendiente
Añadir la clase Tratamientos y sus clases Hijas al codigo de java.	Bautista Miyashiro			Pendiente
Añadir las Funciones de Tratamientos Al crud. (Agregar,editar,Most	Bautista Miyashiro			Pendiente
Añadir Tratamiento al UML y actualizarlo.	Bautista Miyashiro			Pendiente
Completar Las Funcionalidades Extra al Main.	Santiago Cañete			Pendiente
Connvertir El main en un MENU.	Santiago Cañete			Pendiente
BASE DE DATOS				
correciones DER	Matias Basto			Pendiente
Proyecto Informatico 2				
AGREGAR INTERFAZ DE ADMIN	Bautista Miyashiro			Pendiente

3/09/26

Lugar: Laboratorio

Hora: 9:55

Actividad:

No se realizaron actividades el día de hoy

4/09/26

Lugar: Laboratorio

Hora: 9:55

Actividad:

El día de hoy realicé las actividades que me correspondían después de la revisión del 2 de septiembre. Me senté junto a mi compañero Luciano Villarroel y nos pusimos a completar las tareas que nos correspondían, el empezó con la clase turnos. Una vez termino, empecé a hacer mi parte, la cual era añadir las funcionalidades de Turno al CRUD de nuestra clínica privada de POO. Esto me resulto fácil, ya que veníamos en laboratorio practicando este tema, así que lo hice rápido. Al final junto a Luciano terminamos agregándolo al UML, aunque él fue el que se lleva el crédito de eso.

```java
    //++++++++++++++++++++++++{ TURNO }++++++++++++++++++++++++++++++++++++++++//

    //agregar TURNO
    public void Agregar_Turno(Turno t) {
        Lista_Turno.add(t);
    }

    //editar TURNO


public void editarTurno(int IdTurno) {
    Turno tu = buscar_Turno(IdTurno);

    if (tu == null) {
        JOptionPane.showMessageDialog(null, "No se encontró ningún Turno con ese id.");
        return;
    }

    String Nuevo_Codigo_Paciente = JOptionPane.showInputDialog("Ingrese el nuevo código del paciente:", tu.getCodigo_Paciente());
    String Nuevo_nombre_completoMedico = JOptionPane.showInputDialog("Ingrese el nuevo código del médico:", tu.getNombre_completoMedico());
    String Nuevo_Fecha_Hora = JOptionPane.showInputDialog("Ingrese la nueva fecha y hora del turno", tu.getFecha_Hora());
    String Nuevo_Especialidad = JOptionPane.showInputDialog("Ingrese la nueva especialidad:", tu.getEspecialidad());
    int Nueva_sala = Integer.parseInt(JOptionPane.showInputDialog("Ingrese la sala de consultorio:", tu.getSala()));
    String Nuevo_Estado = JOptionPane.showInputDialog("Ingrese el estado del turno (Pendiente/Confirmado/Cancelado/Atendido):", tu.getEstado());
    String Nuevo_MotivoConsulta = JOptionPane.showInputDialog("Ingrese el motivo de la consulta:", tu.getMotivoConsulta());
    String Nuevo_TipoTurno = JOptionPane.showInputDialog("Ingrese el tipo de turno :", tu.getTipoTurno());


    tu.setCodigo_Paciente(Nuevo_Codigo_Paciente);
    tu.setNombre_completoMedico(Nuevo_nombre_completoMedico);
    tu.setFecha_Hora(Nuevo_Fecha_Hora);
    tu.setEspecialidad(Nuevo_Especialidad);
    tu.setSala(Nueva_sala);

    tu.setEstado(Nuevo_Estado);
    tu.setMotivoConsulta(Nuevo_MotivoConsulta);
```

Output ×

```java
    //ELIMINAR TURNO

    public void Eliminar_Turno(int IdTurno){
       Turno tu = buscar_Turno(IdTurno);

       if (tu !=null){
          Lista_Turno.remove(tu);
          JOptionPane.showMessageDialog(null, "Turno" + IdTurno + " Eliminado con exito.");
       }else{
          JOptionPane.showMessageDialog(null, "Turno con el ID " + IdTurno + " no encontrado.");
       }


    }

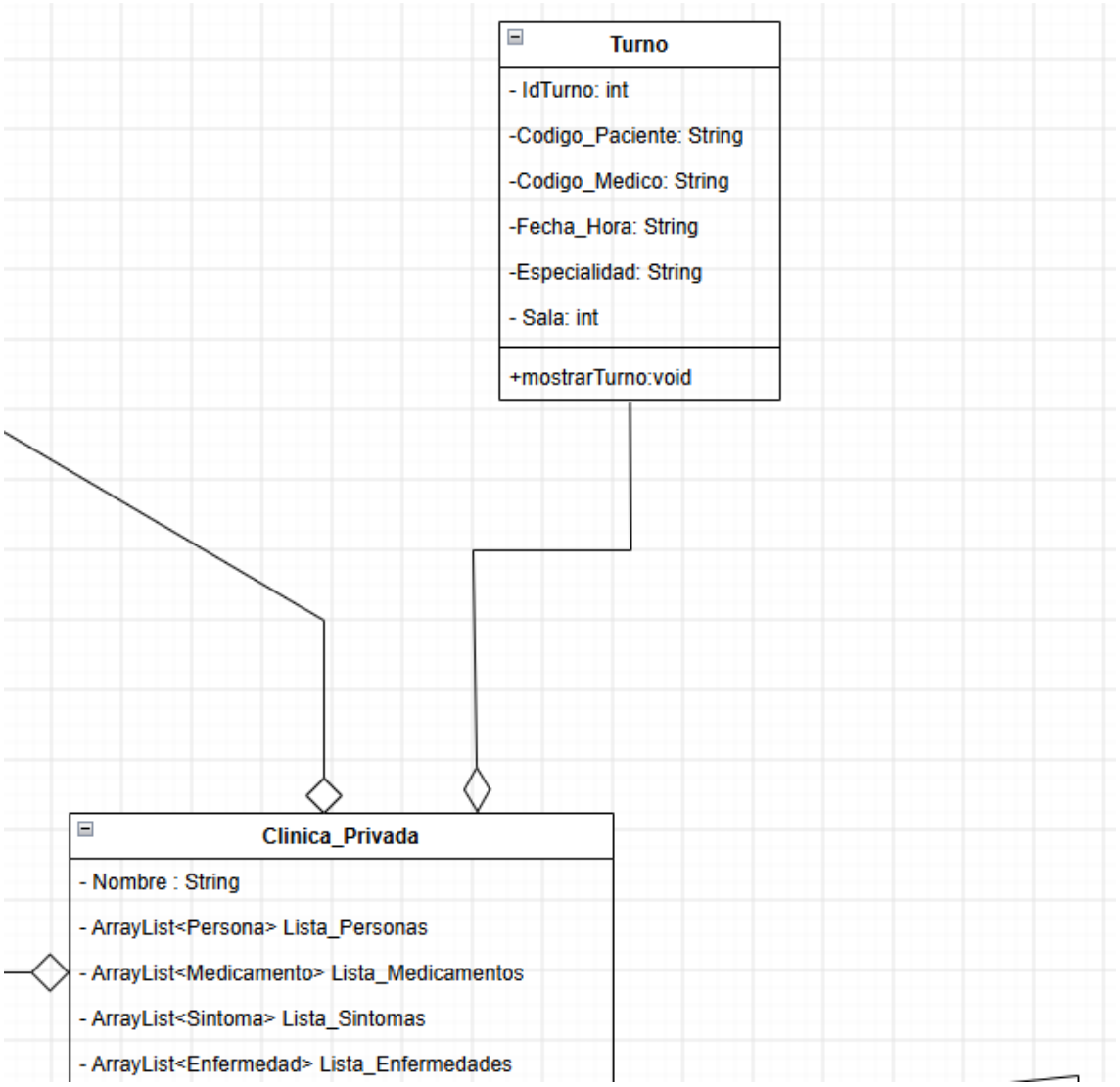
   //BUSCAR TURNO

    public Turno buscar_Turno(int IdTurno){
        for(Turno tu : Lista_Turno){
            if(tu.getIdTurno() == IdTurno){
                tu.mostrarTurno();
                return tu;
            }
        }

        return null;
    }

  //MOSTRAR TURNO

    public void mostrarTodosLosTurnos() {
    if (Lista_Turno.isEmpty()) {
        JOptionPane.showMessageDialog(null, "No hay turnos registrados.");
        return;
    }
    for (Turno tu : Lista_Turno) {
        tu.mostrarTurno();
```



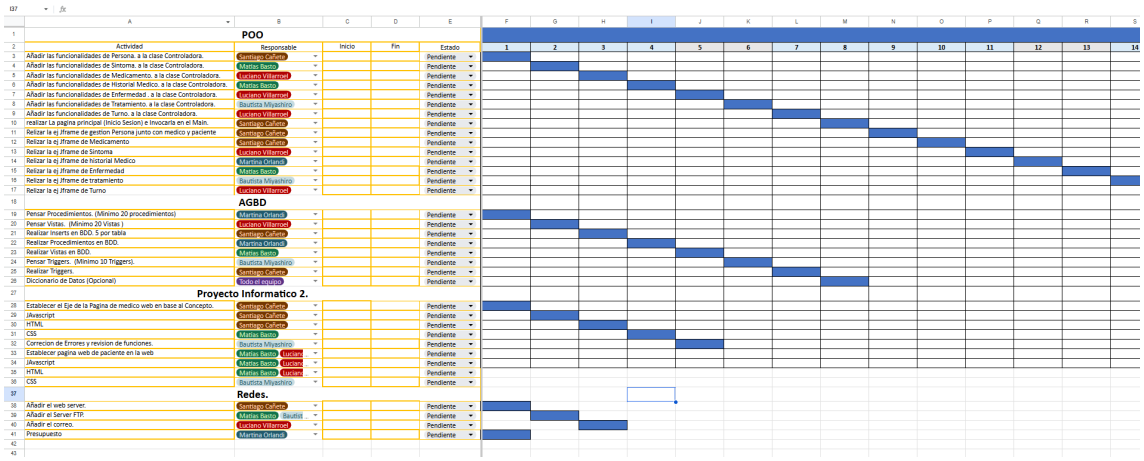
07/09/26

Lugar: Laboratorio

Hora: 9:55

Actividad:

El día de hoy hable con el líder para plantear las nuevas actividades y plasmarlo en el Gantt, decidimos seguir los consejos del profe y separarnos las actividades de manera que todos participen al menos un poco



	Actividad	Responsable	Inicio	Fin	Estado
	POO				
3	Añadir las funcionalidades de Persona. a la clase Controladora.	Santiago Cañete			Pendiente
4	Añadir las funcionalidades de Sintoma. a la clase Controladora.	Matias Bello			Pendiente
5	Añadir las funcionalidades de Medicamento. a la clase Controladora.	Luciano Villarroel			Pendiente
6	Añadir las funcionalidades de Historial Medico. a la clase Controladora.	Matias Bello			Pendiente
7	Añadir las funcionalidades de Enfermedad . a la clase Controladora.	Luciano Villarroel			Pendiente
8	Añadir las funcionalidades de Tratamiento. a la clase Controladora.	Bautista Miyashiro			Pendiente
9	Añadir las funcionalidades de Turno. a la clase Controladora.	Luciano Villarroel			Pendiente
10	realizar La pagina principal (Inicio Sesion) e invocarla en el Main.	Santiago Cañete			Pendiente
11	Relizar la ej Jframe de gestion Persona junto con medico y paciente	Santiago Cañete			Pendiente
12	Relizar la ej Jframe de Medicamento	Santiago Cañete			Pendiente
13	Relizar la ej Jframe de Sintoma	Luciano Villarroel			Pendiente
14	Relizar la ej Jframe de historial Medico	Martina Orlandi			Pendiente
15	Relizar la ej Jframe de Enfermedad	Matias Bello			Pendiente
16	Relizar la ej Jframe de tratamiento	Bautista Miyashiro			Pendiente
17	Relizar la ej Jframe de Turno	Luciano Villarroel			Pendiente
18	**AGBD**				
19	Pensar Procedimientos. (Minimo 20 procedimientos)	Martina Orlandi			Pendiente
20	Pensar Vistas. (Minimo 20 Vistas)	Luciano Villarroel			Pendiente
21	Realizar Inserts en BDD. 5 por tabla	Santiago Cañete			Pendiente
22	Realizar Procedimientos en BDD.	Martina Orlandi			Pendiente
23	Realizar Vistas en BDD.	Matias Bello			Pendiente
24	Pensar Triggers. (Minimo 10 Triggers).	Bautista Miyashiro			Pendiente
25	Realizar Triggers.	Santiago Cañete			Pendiente
26	Diccionario de Datos (Opcional)	todo el equipo			Pendiente
27	**Proyecto Informatico 2.**				
28	Establecer el Eje de la Pagina de medico web en base al Concepto.	Santiago Cañete			Pendiente
29	JAvascript	Santiago Cañete			Pendiente
30	HTML	Santiago Cañete			Pendiente
31	CSS	Matias Bello			Pendiente
32	Correccion de Errores y revision de funciones.	Bautista Miyashiro			Pendiente
33	Establecer pagina web de paciente en la web	Matias Bello Luciano			Pendiente
34	JAvascript	Matias Bello Luciano			Pendiente
35	HTML	Matias Bello Luciano			Pendiente
36	CSS	Bautista Miyashiro			Pendiente
37	**Redes.**				
38	Añadir el web server.	Santiago Cañete			Pendiente
39	Añadir el Server FTP.	Matias Bello Bautist			Pendiente
40	Añadir el correo.	Luciano Villarroel			Pendiente
41	Presupuesto	Martina Orlandi			Pendiente

08/09/26

Lugar: Laboratorio

Hora: 9:55

Actividad:

No se realizaron actividades el día de hoy

09/09/26

Lugar: Aula programación

Hora: 9:55

Actividad:

No se realizaron actividades el día de hoy

10/09/26

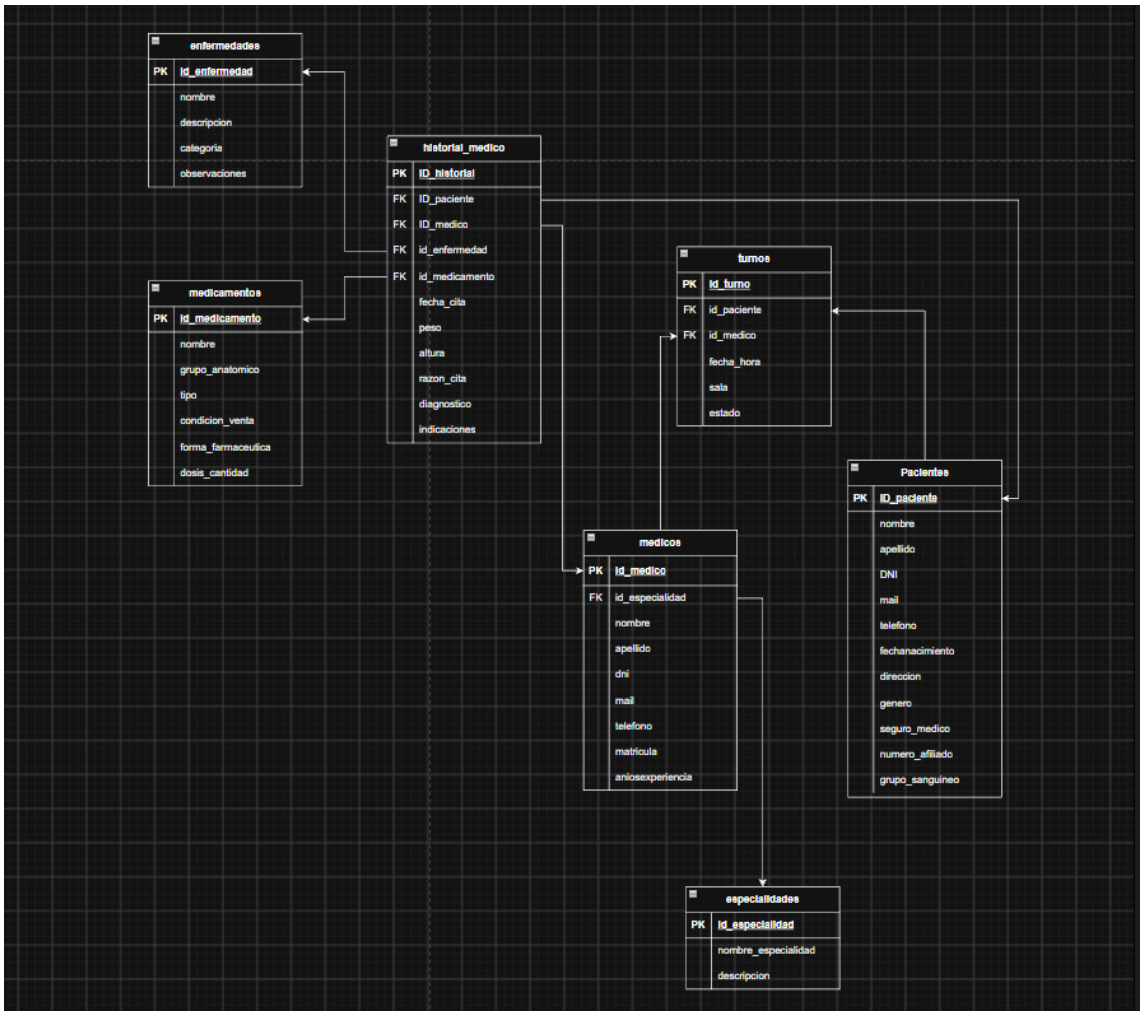
Lugar: Laboratorio

Hora: 9:55

Actividad:

El día de hoy tras la revisión pasada, decidí empezar a modificar la base de datos y el DER que me pidieron los profesores que modifiqué, en base a lo

que me pidieron hice una modificación en primera instancia. Elimine la tabla síntomas momentáneamente y agregue la tabla de especialidades



En especialidad agregue los atributos de id, nombre y descripción

```sql
drop database if exists clinica_privada;
create database clinica_privada;
use clinica_privada;

create table especialidades (
    id_especialidad int auto_increment primary key,
    nombre_especialidad varchar(100),
    descripcion text
);
create table medicos (
    id_medico int primary key auto_increment,
    nombre varchar(50) not null,
    apellido varchar(50) not null,
    dni varchar(20) unique not null,
    mail varchar(50),
    telefono varchar(50),
    matricula varchar(20) unique not null,
    aniosexperiencia int,
    id_especialidad int,
    foreign key (id_especialidad) references especialidades(id_especialidad)
);

create table pacientes (
    id_paciente int primary key auto_increment,
    nombre varchar(50),
    apellido varchar(50),
    dni varchar(20),
    mail varchar(50),
    telefono varchar(50),
    fechanacimiento date,
    direccion varchar(100),
    genero varchar(20),
    seguro_medico varchar(50),
```

```sql
create table turnos (
    id_turno int auto_increment primary key ,
    id_paciente int not null,
    id_medico int not null,
    fecha_hora datetime,
    sala varchar(20),
    estado varchar(30),
    foreign key (id_paciente) references pacientes(id_paciente),
    foreign key (id_medico) references medicos(id_medico)
);

create table medicamentos (
    id_medicamento int auto_increment primary key ,
    nombre varchar(100),
    grupo_anatomico varchar(100),
    tipo varchar(50),
    condicion_venta varchar(50),
    forma_farmaceutica varchar(50),
    dosis_cantidad varchar(50)
);

create table enfermedades (
    id_enfermedad int auto_increment primary key ,
    nombre varchar(100),
    descripcion text,
    categoria varchar(50),
    observaciones text
);

create table historial_medico (
    id_historial int primary key auto_increment,
    id_paciente int not null,
```

```
    id_medico int not null,
id_enfermedad int,
    id_medicamento int,
    fecha_cita datetime,
    peso decimal(5,2),
    altura decimal(5,2),
    razon_cita text,
    diagnostico text,
    indicaciones text,
    foreign key (id_paciente) references pacientes(id_paciente),
    foreign key (id_medico) references medicos(id_medico),
    foreign key (id_enfermedad) references enfermedades(id_enfermedad),
    foreign key (id_medicamento) references medicamentos(id_medicamento)
);
```

Finalmente, esta es la nueva base de datos modificada el día de hoy

11/09/26

Lugar: Laboratorio

Hora: 9:55

Actividad:

No se realizaron actividades el día de hoy

14/09/26

Lugar: Laboratorio

Hora: 9:55

Actividad:

No se realizaron actividades a la fecha

15/09/26

Lugar: Laboratorio

Hora: 9:55

Actividad:

No se realizaron actividades a la fecha

16/09/26

Lugar: Aula Informática

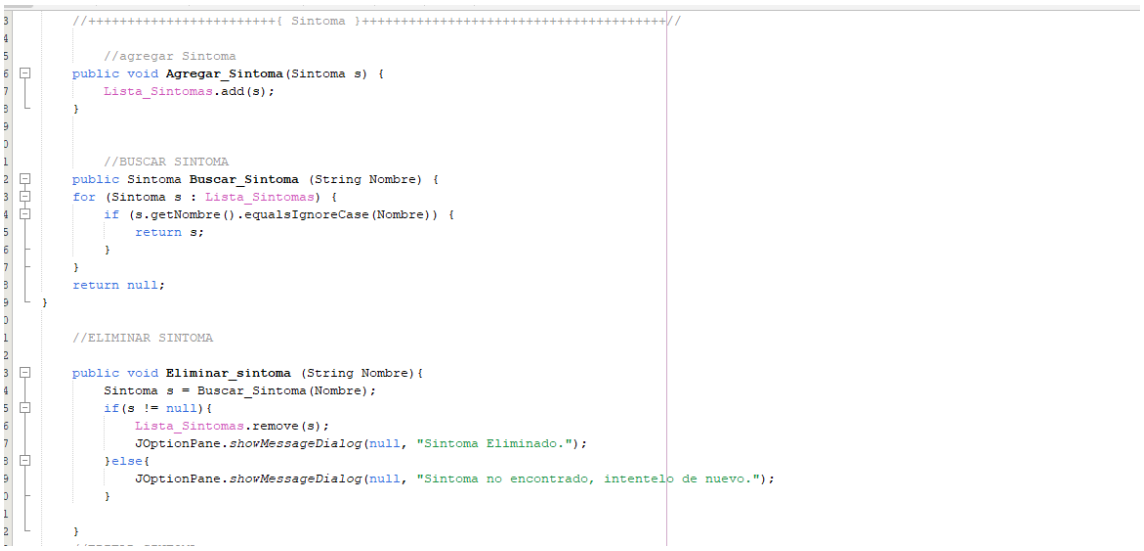
Hora: 9:55

Actividad:

El día de hoy seguí con las tareas que se me asignaron

El día de hoy seguí realizando las actividades plasmadas al Gantt, me asignaron la parte de agregar las funcionalidades de Síntoma e historial médico a la clase controlador de nuestro proyecto en la parte de POO y hacer el JFrame de enfermedad. Empecé agregando solamente con las funcionalidades de las 2 clases que me asignaron, nuevamente no encontré dificultad ya que veníamos practicando esto. Las funciones que agregué fueron: Agregar Síntoma/Historial médico – Buscar Síntoma/Historial médico – Eliminar Síntoma/Historial médico – Buscar Síntoma/Historial médico – Editar Síntoma/Historial médico

FUNCIONES DE SINTOMAS



```
    //++++++++++++++++++++++{ Sintoma }++++++++++++++++++++++++++++++++++++++//

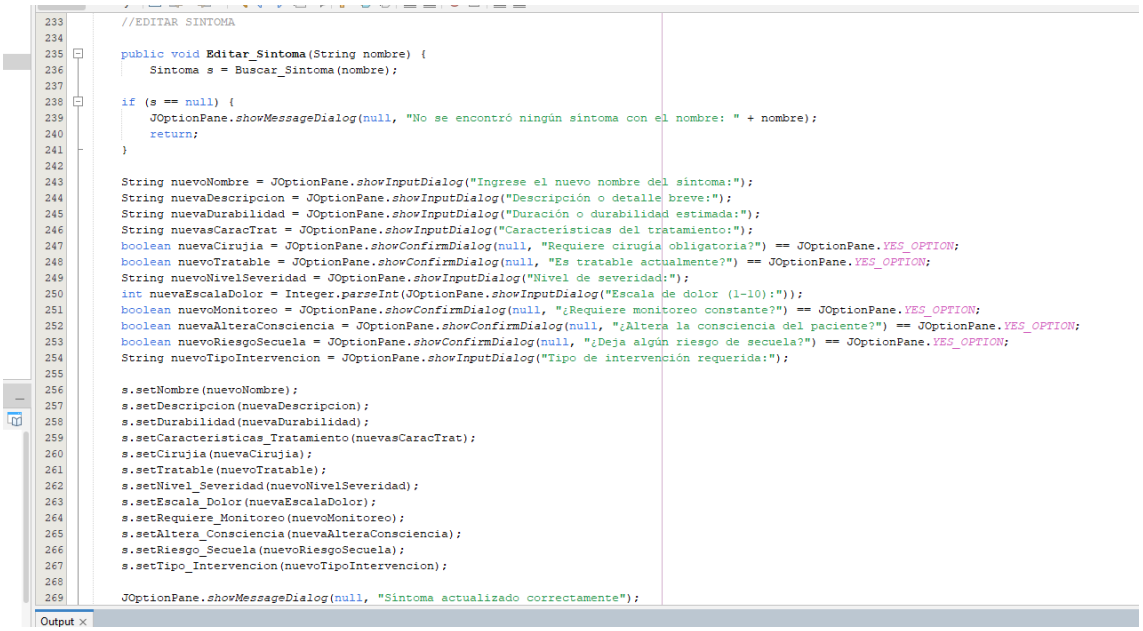
        //agregar Sintoma
    public void Agregar_Sintoma(Sintoma s) {
        Lista_Sintomas.add(s);
    }


        //BUSCAR SINTOMA
    public Sintoma Buscar_Sintoma (String Nombre) {
    for (Sintoma s : Lista_Sintomas) {
        if (s.getNombre().equalsIgnoreCase(Nombre)) {
            return s;
        }
    }
    return null;
}

    //ELIMINAR SINTOMA

    public void Eliminar_sintoma (String Nombre){
        Sintoma s = Buscar_Sintoma(Nombre);
        if(s != null){
            Lista_Sintomas.remove(s);
            JOptionPane.showMessageDialog(null, "Sintoma Eliminado.");
        }else{
            JOptionPane.showMessageDialog(null, "Sintoma no encontrado, intentelo de nuevo.");
        }

    }
```



```
    //EDITAR SINTOMA

    public void Editar_Sintoma(String nombre) {
        Sintoma s = Buscar_Sintoma(nombre);

    if (s == null) {
        JOptionPane.showMessageDialog(null, "No se encontró ningún síntoma con el nombre: " + nombre);
        return;
    }

    String nuevoNombre = JOptionPane.showInputDialog("Ingrese el nuevo nombre del síntoma:");
    String nuevaDescripcion = JOptionPane.showInputDialog("Descripción o detalle breve:");
    String nuevaDurabilidad = JOptionPane.showInputDialog("Duración o durabilidad estimada:");
    String nuevasCaracTrat = JOptionPane.showInputDialog("Características del tratamiento:");
    boolean nuevaCirujia = JOptionPane.showConfirmDialog(null, "Requiere cirugía obligatoria?") == JOptionPane.YES_OPTION;
    boolean nuevoTratable = JOptionPane.showConfirmDialog(null, "Es tratable actualmente?") == JOptionPane.YES_OPTION;
    String nuevoNivelSeveridad = JOptionPane.showInputDialog("Nivel de severidad:");
    int nuevaEscalaDolor = Integer.parseInt(JOptionPane.showInputDialog("Escala de dolor (1-10):"));
    boolean nuevoMonitoreo = JOptionPane.showConfirmDialog(null, "¿Requiere monitoreo constante?") == JOptionPane.YES_OPTION;
    boolean nuevaAlteraConsciencia = JOptionPane.showConfirmDialog(null, "¿Altera la consciencia del paciente?") == JOptionPane.YES_OPTION;
    boolean nuevoRiesgoSecuela = JOptionPane.showConfirmDialog(null, "¿Deja algún riesgo de secuela?") == JOptionPane.YES_OPTION;
    String nuevoTipoIntervencion = JOptionPane.showInputDialog("Tipo de intervención requerida:");

    s.setNombre(nuevoNombre);
    s.setDescripcion(nuevaDescripcion);
    s.setDurabilidad(nuevaDurabilidad);
    s.setCaracteristicas_Tratamiento(nuevasCaracTrat);
    s.setCirujia(nuevaCirujia);
    s.setTratable(nuevoTratable);
    s.setNivel_Severidad(nuevoNivelSeveridad);
    s.setEscala_Dolor(nuevaEscalaDolor);
    s.setRequiere_Monitoreo(nuevoMonitoreo);
    s.setAltera_Consciencia(nuevaAlteraConsciencia);
    s.setRiesgo_Secuela(nuevoRiesgoSecuela);
    s.setTipo_Intervencion(nuevoTipoIntervencion);

    JOptionPane.showMessageDialog(null, "Síntoma actualizado correctamente");
```

Output

FUNCIONES DE HISTORIAL MEDICO

17/09/26

Lugar: Laboratorio

Hora: 9:55

Actividad:

18/09/26

Lugar: Aula Informática

Hora: 9:55

Actividad:

21/09/26

Lugar: Laboratorio

Hora: 9:55

Actividad:

22/09/26

Lugar: Laboratorio

Hora: 9:55

Actividad:

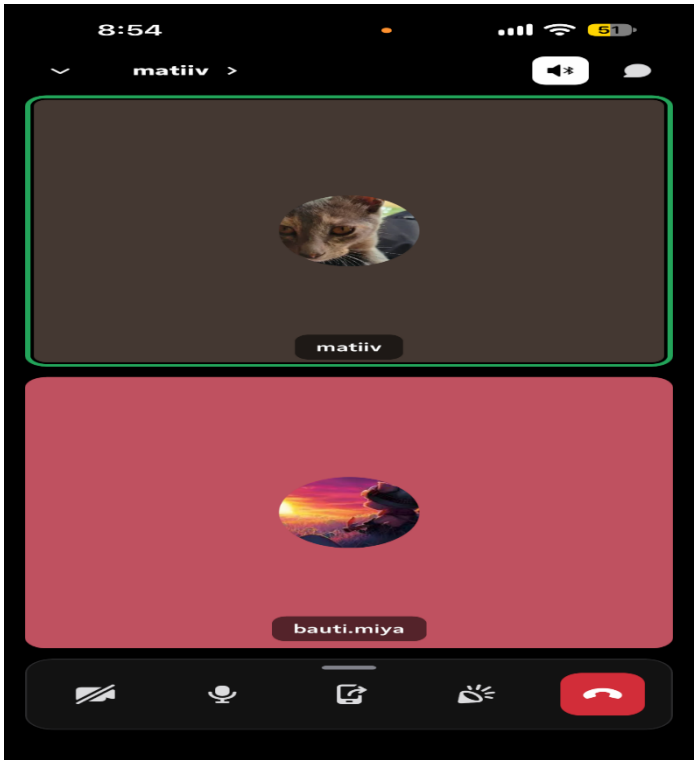
23/09/26

Lugar: Aula Informática

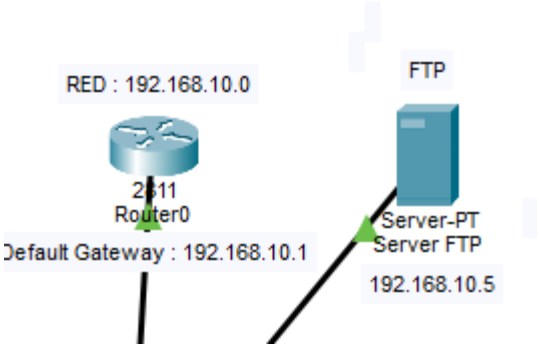
Hora: 9:55

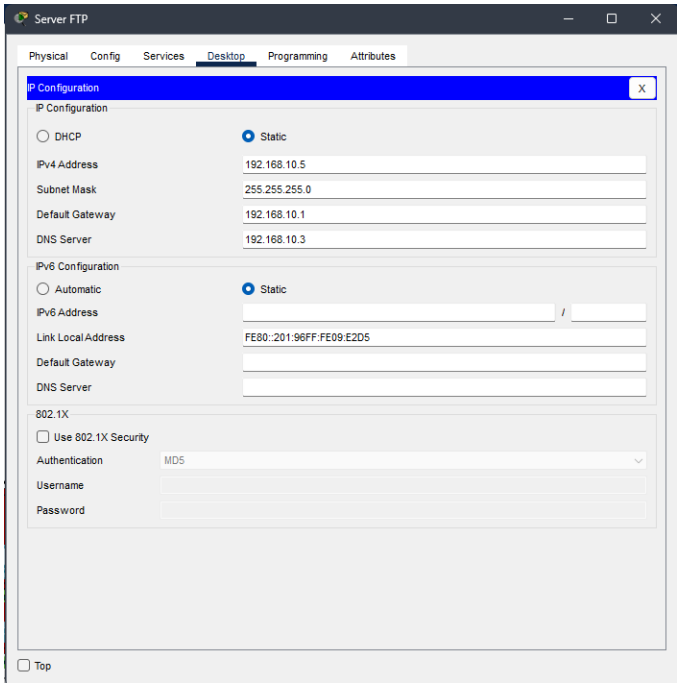
Actividad:

El día de hoy me puse a ver el Gantt para seguir realizando las tareas pendientes que me quedaban. En el Gantt marcaba que tenía que realizar el servidor FTP junto a mi compañero Bautista Miyashiro. Le escribí para coordinar como íbamos a hacer nuestra actividad o cuando la empezaríamos y me comento que podíamos empezarla este mismo día. Así que le dije que hagamos llamada por Discord y lo hacíamos desde su compu, ya que a los dos nos va bien en redes, no nos cuesta tanto.

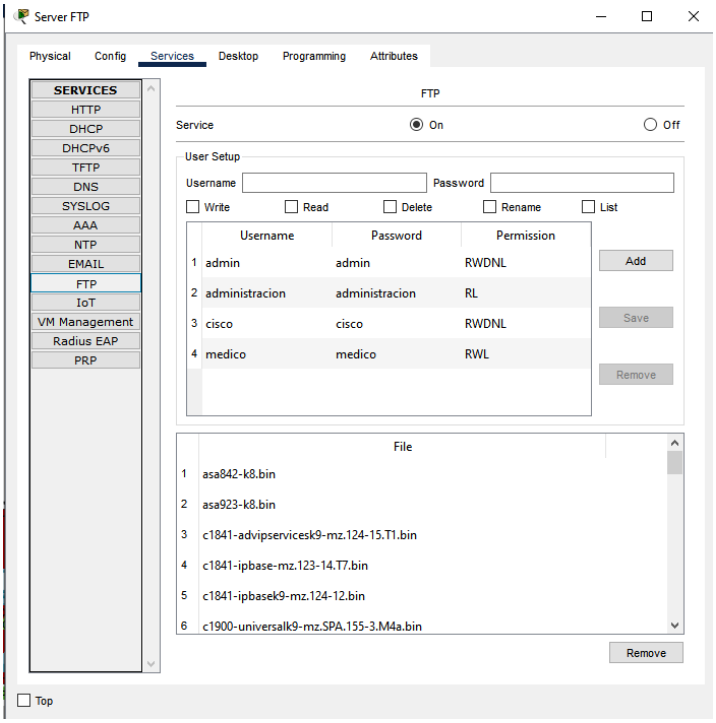


Entramos a una llamada y nos pusimos manos a la obra





Hicimos la configuración del servidor de manera impecable, no nos tiro error, pero a la hora de configurar el FTP, ninguno se acordaba como hacerlo, así que recurrimos al classroom de nuestro profesor de redes Nahuel Acosta. Una vez lo leímos, pudimos terminar de configurar el servidor



24/09/26

Lugar: Laboratorio

Hora: 9:55

Actividad:

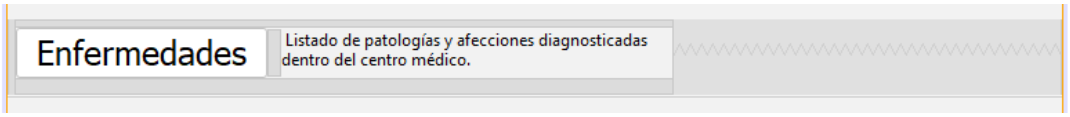
25/09/26

Lugar: Laboratorio

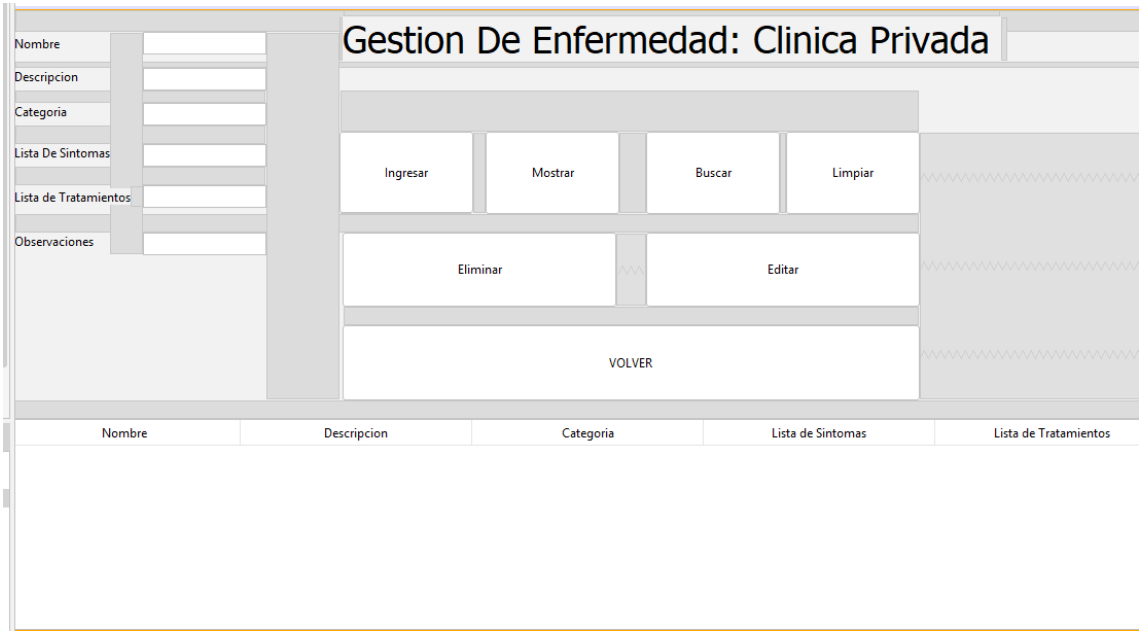
Hora: 9:55

Actividad:

El día de hoy estamos a nada de entregar el proyecto para la 3ra revisión del proyecto, a todo esto, continue con mi parte de POO, después de las funciones que había hecho anteriormente, me toca hacer el Jframe de enfermedades, así que empecé a completarla. Como primer paso cree el acceso desde la selección de gestión a la gestión de enfermedad.



Después de eso, continúe con la creación del Jframe, realicé la elaboración de los botones, txt, tabla y todo lo estético del Jframe



Después como ultimo paso pase a realizar el funcionamiento de los botones con las funciones que hice anteriormente en la clase controladora.

```
private void btnIngresarActionPerformed(java.awt.event.ActionEvent evt) {
  String Nombre = txtNombre.getText();
    String Descripcion = txtDescripcion.getText();
    String Categoria = txtCategoria.getText();
    String Lista_sintomas = txtListaSintomas.getText();
    String Lista_tratamientos = txtListaTratamientos.getText();
    String Observaciones = txtObservaciones.getText();

    Enfermedad nuevaEnfermedad = new Enfermedad(0, Nombre, Descripcion, Categoria, Lista_sintomas, Lista_tratamientos, Observaciones);
    CP.AgregarEnfermedad(nuevaEnfermedad);

    JOptionPane.showMessageDialog(null, "Enfermedad registrada exitosamente !");
    txtNombre.setText("");
    txtDescripcion.setText("");
    txtCategoria.setText("");
    txtListaSintomas.setText("");
    txtListaTratamientos.setText("");
    txtObservaciones.setText("");

}

private void btnMostrarActionPerformed(java.awt.event.ActionEvent evt) {
 actualizarTabla();
}

private void btnEliminarActionPerformed(java.awt.event.ActionEvent evt) {
    String nombre = txtNombre.getText();
    CP.Eliminar_Enfermedad(nombre);
}

private void btnEditarActionPerformed(java.awt.event.ActionEvent evt) {
CP.editarEnfermedad(txtNombre.getText());
}
```

```
private void btnBuscarActionPerformed(java.awt.event.ActionEvent evt) {
String nombreBuscado = txtNombre.getText();
Enfermedad e = CP.Buscar_Enfermedad(nombreBuscado);
if (e != null) {
    mostrarEnfermedadTabla(e);
} else {
    JOptionPane.showMessageDialog(null, "No hay ninguna enfermedad con ese nombre.");
}
}

private void btnLimpiarActionPerformed(java.awt.event.ActionEvent evt) {
    txtNombre.setText("");
    txtDescripcion.setText("");
    txtCategoria.setText("");
    txtListaSintomas.setText("");
    txtListaTratamientos.setText("");
    txtObservaciones.setText("");

}

     public void mostrarEnfermedadTabla(Enfermedad e){
    DefaultTableModel modelo = (DefaultTableModel)tablaEnfermedad.getModel();
    modelo.setRowCount(0);

    Object [] fila ={
                e.getNombre(),
                e.getDescripcion(),
                e.getCategoria(),
                e.getLista_Sintomas(),
                e.getLista_Tratamientos(),
                e.getObservaciones(),
            };
    modelo.addRow(fila);

}
    private void actualizarTabla(){
```

Además, agregué nuevas vistas y procedimientos pensados con mi compañero Luciano Villarroel

```
+   vista_turnos_Asignados
+   vista_Enfermedad_Paciente
+   vista_Medicamentos_
```

```
+   buscar_mediacamento_por_id
+   registrar_enfermedades
+   registrar_turno
+   cancelar_turno
+   obtener_turnos_del_dia
+   reporte_medicos_por_especialidad
+   calcular_imc_paciente
+   ver_recetas_paciente
+   cambiar_sala_turno
+   top_enfermedades_diagnosticadas
+   buscar_medicos_por_especialidad
```

Una vez terminé, lo subí al GitHub y así finalicé el día.

26/09/26

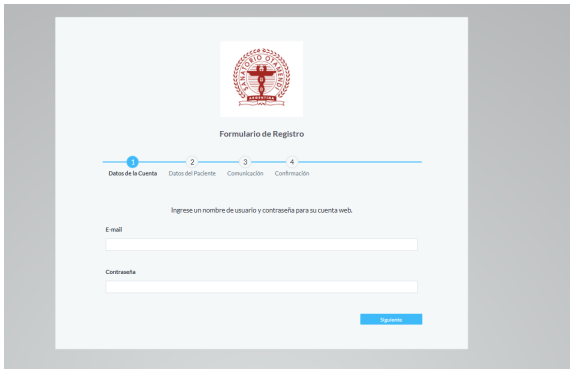
Lugar: Laboratorio

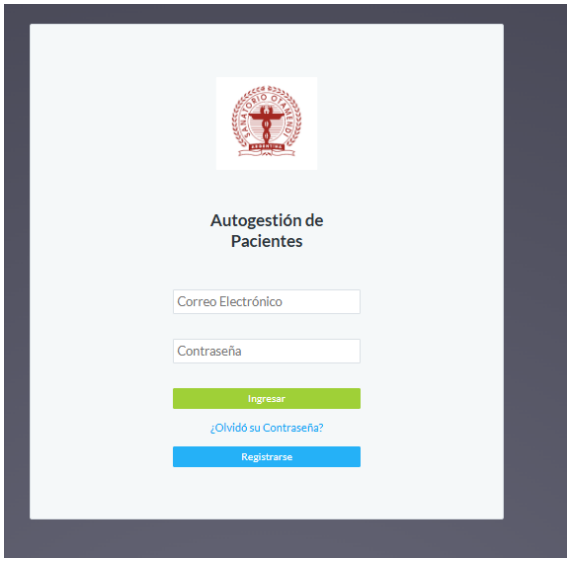
Hora: 9:55

Actividad:

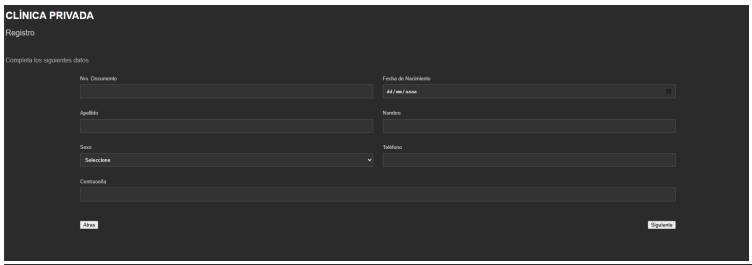
El día de hoy hice una llamada con mi compañero Luciano Villarroel para continuar con la página web antes de la entrega del lunes, hablamos anteriormente y nos dijo que había hecho algo del html, pero que faltaban cosas, así que descargamos el archivo y vimos que ya tenia un index, con el titulo simple y los botones de registrarse e iniciar sesión dirigiéndote a una

página en negro, ya que todavía no había nada. Antes de empezar nos quisimos dar una idea de como lo haríamos, asi que buscamos en Google y nos inspiramos principalmente en el hospital Otamendi



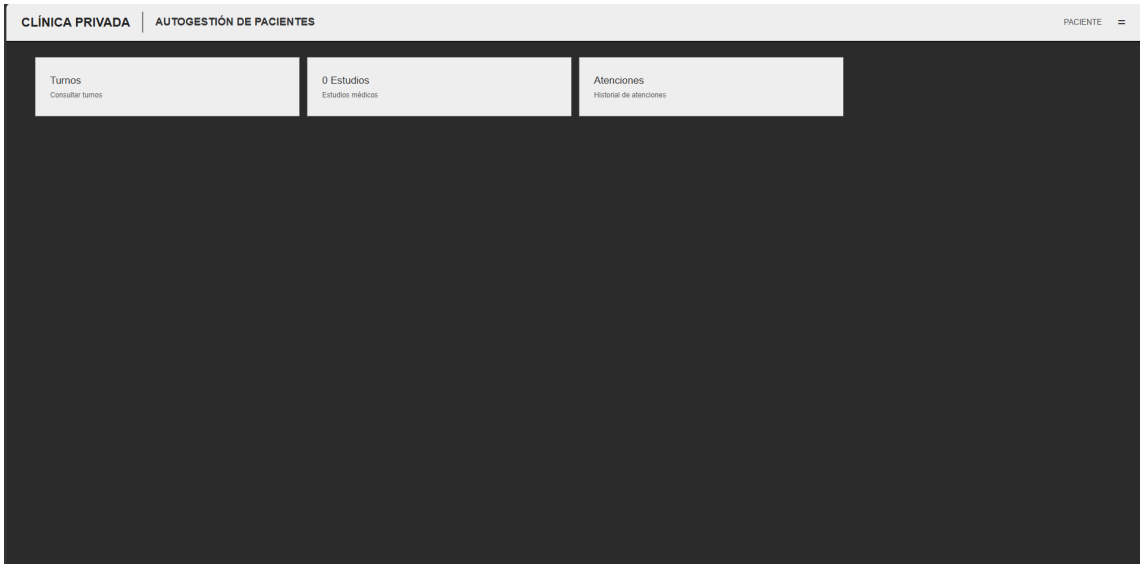


Como primero armamos un formulario de registro con las etiquetas div, h1, h2, p, form, label, input, select y boton. Con div organicé el contenedor y las filas de campos, y con input (tipos text, date y contraseña) y select cargué los siete datos: documento, fecha de nacimiento, apellido, nombre, sexo, teléfono y contraseña. Agregué los botones “Atrás” y “Siguiente”. Con script escribí el código que valida que no haya campos vacíos y muestra un alert si falta alguno. Los estilos los vinculé con link a un CSS externo y quedó así





Por ultimo Armé la pantalla de gestión del paciente usando las etiquetas div, h2, p y button. Con div armé la barra superior, que lleva el nombre de la clínica, un separador, el texto “Autogestión de pacientes”, el usuario “Paciente” y un boton de menú. Debajo armé un panel con tres opciones (Turnos, Estudios y Atenciones). Cada una tiene un div con un ícono, un h2 como título y un p como descripción. Finalmente quedando así



Asi finalizando por este día la página

28/09/26

Lugar: Laboratorio

Hora: 9:55

Actividad:

El día de hoy llego la 3ra revisión de nuestro proyecto, como siempre, los profesores nos pidieron que acomodáramos todo lo que teníamos hasta ahora en las computadoras. Lamentablemente por problemas de salud me tuve que retirar al hospital antes de la revisión de nuestro proyecto. De todas formas, le escribí al líder mi grupo para preguntarle como nos había ido y me dijo que los profesores nos dijeron que estaba todo bien, no nos faltaba nada, que les alegra como nos estamos distribuyendo las tareas y que sigamos así.

29/09/26

Lugar: Laboratorio

Hora: 9:55

Actividad:

No se realizaron actividades el día debido a un reposo con certificado del hospital de 1 semana

30/09/26

Lugar: Aula informática

Hora: 9:55

Actividad:

No se realizaron actividades el día debido a un reposo con certificado del hospital de 1 semana

1/10/26

Lugar: Laboratorio

Hora: 9:55

Actividad:

No se realizaron actividades el día debido a un reposo con certificado del hospital de 1 semana

02/10/26

Lugar: Laboratorio

Hora: 9:55

Actividad:

No se realizaron actividades el día debido a un reposo con certificado del hospital de 1 semana

05/10/26

Lugar: Laboratorio

Hora: 9:55

Actividad:

El día de hoy plasmamos las consignas que nos tocaban

No se realizaron actividades el día debido a un reposo con certificado del hospital de 1 semana

06/10/26

Lugar: Laboratorio

Hora: 9:55

Actividad:

No se realizaron actividades el día debido a un reposo con certificado del hospital de 1 semana

07/10/26

Lugar: Aula Informática

Hora: 9:55

Actividad:

No se realizaron actividades el día de hoy

08/10/26

Lugar: Laboratorio

Hora: 9:55

Actividad:

09/10/26

Lugar: Laboratorio

Hora: 9:55

Actividad

12/10/26

Lugar: Laboratorio

Hora: 9:55

Actividad

13/10/26

Lugar: Laboratorio

Hora: 9:55

Actividad

14/10/26

Lugar: Laboratorio

Hora: 9:55

Actividad

15/10/26

Lugar: Laboratorio

Hora: 9:55

Actividad

16/10/26

Lugar: Laboratorio

Hora: 9:55

Actividad

19/10/26

Lugar: Laboratorio

Hora: 9:55

Actividad

El día de hoy continue con mis deberes plasmados en el Gantt, me toca realizar la creación de los algunos DAO para conectar los JFrame y sus funciones a nuestra Base de Datos. Primero empecé con comparar la clase de Medicamentos y Turnos con la del base de datos. Como había cosas que no me sonaban mucho, decidí modificarlo y acoplarla a como me parecería que seria funcional en cuanto a agregar un turno y un medicamento.



```sql
create table turnos (
    id_turno int auto_increment primary key ,
    id_paciente int not null,
    id_medico int not null,
    fecha_hora varchar(50),
    Especialidad varchar(30),
    sala int,
    foreign key (id_paciente) references paciente(id_paciente)
);

create table medicamentos (
    id_medicamento int auto_increment primary key ,
    nombre varchar(100),
    grupo_anatomico varchar(100),
    tipo varchar(50),
    condicion_venta varchar(50),
    forma_farmaceutica varchar(50),
    Categoria_Subtipo varchar(50),
    Via_Administracion varchar(50),
    Dosis_Unidad varchar(50),
    dosis_cantidad varchar(50)
);
```



```sql
BaseDeDatosJava.sql

@@ -8,7 +8,7 @@ create table pacientes (
    id_paciente int primary key auto_increment,
    Nombre varchar(50),
    Apellido varchar(50),
-   DNI varchar(20),
+   DNI int unique,
    Email varchar(50),
    Fecha_Nacimiento varchar(100),
    Direccion varchar(100),
@@ -23,7 +23,7 @@ create table medicos (
    id_medico int primary key auto_increment,
    Nombre varchar(50),
    Apellido varchar(50),
-   DNI varchar(20),
+   DNI int unique,
    Email varchar(50),
    Fecha_Nacimiento varchar(50),
    Direccion varchar(100),
@@ -32,7 +32,7 @@ create table medicos (
    Numero_Matricula varchar(50),
    Numero_Matricula varchar(10) unique,
    Años_Experiencia int
-   );
+   );

   create table turnos (
    id_turno int auto_increment primary key ,
-   id_paciente int not null,
-   id_medico int not null,
+   codigo_paciente int,
+   nombre_completoMedico varchar(100),
    fecha_hora varchar(50),
+   matricula_medico varchar(30),
    Especialidad varchar(30),
    sala int,
-   foreign key (id_paciente) references paciente(id_paciente)
+   estado varchar(20),
+   motivoConsulta varchar(255),
+   tipoTurno varchar(50),
+   foreign key (matricula_medico) references medicos(Numero_Matricula)
+
    );
```



```sql
create table enfermeros (

@@ -10,14 +11,17 @@ create table Usuarios(

);

create table Sintomas(
-   id_sintoma int auto_increment primary key,
-   Nombre varchar(255),
+   Nombre varchar(100) primary key,
    Descripcion varchar(255),
    Duracionhoras varchar(100),
    Caracteristicas_Tratamiento varchar(100),
-   Clinica varchar(20),
-   Tratable varchar(20),
+   Clinica boolean,
+   Tratable boolean,
    Nivel_Gravedad varchar(50),
    Escala_Dolor int,
-   Respira_Monitores varchar(20),
-   Alivia_Consciencia varchar(20),
-   Riesgo_Secuela varchar(20),
+   Respira_Monitores boolean,
+   Alivia_Consciencia boolean,
+   Riesgo_Secuela boolean,
    Tipo_Intervencion varchar(100)
);

@@ -113,9 +117,7 @@ INSERT INTO Sintomas (Nombre, Descripcion, Duracionhoras, Caracteristicas_Tratam
('Fractura expuesta', 'Fractura con comunicación entre el hueso y el exterior', 'Semanas a meses', 'Estabilización traumatológica y estabilización', 'Sí', 'Sí', 'Grave', 0, 'Sí', 'No', 'Sí', 'Quirúrgico'),
('Tos seca', 'Tos sin producción significativa de secreción', 'Variable', 'Evaluación de la causa y tratamiento según indicación médica', 'No', 'Sí', 'Leve', 0, 'No', 'No', 'No', 'Variable');

-
+

@@ -133,9 +135,9 @@ INSERT INTO pacientes (Nombre, Apellido, DNI, Email, Fecha_Nacimiento, Direccion
('Martín', 'González', '38421518', 'gonzalezmartin98@gmail.com', '1998-04-14', 'Av Santa Fe 2345 4B', 'Masculino', 'O0RH', '216-44387-1', 'No', 'Hipertensión, operado de apendicitis en 2014'),
('Florencia', 'Rodríguez', '41235894', 'flor.rodriguez@hotmail.com', '2000-12-05', 'Piedrabuena 852', 'Femenino', 'Sello Medical', 'OM-441239', 'No', 'Nada crónico de nacimiento. Due perf.'),
@@ -151,15 +153,15 @@ VALUES
(5, 'Federico', 'Ramírez', '23675543', 'federico.ramirez@gmail.com', '1982-05-30', 'Sarmiento 1580', 'Masculino', 'Dermatología', 'PA-58787', 12);

-
+

INSERT INTO turnos
- (id_turno, id_paciente, id_medico, fecha_hora, Especialidad, sala)
+ (id_turno, codigo_paciente, codigo_medico, fecha_hora, Especialidad, sala, estado, motivoConsulta, tipoTurno, observaciones)
VALUES
- (1, 6, 3, '2024-10-12 09:00:00', 'Cardiología', 101),
- (2, 8, 5, '2024-10-12 10:30:00', 'Pediatría', 102),
- (3, 9, 7, '2024-10-13 11:00:00', 'Clínica médica', 103),
- (4, 6, 9, '2024-10-13 14:00:00', 'Traumatología', 104),
- (5, 8, 5, '2024-10-14 15:30:00', 'Dermatología', 105);
+ (1, '38421518', '3', '2024-10-12 09:00:00', 'Cardiología', 'Confirmado', 'Control general', 'Presencial', 'Ninguna'),
+ (2, '41235894', '5', '2024-10-12 10:30:00', 'Pediatría', 'Pendiente', 'Presencial', 'Traer libreta'),
+ (3, '38421518', '7', '2024-10-13 11:00:00', 'Clínica médica', 'Confirmado', 'Dolor de estómago', 'Urgencia', 'En ayunas'),
+ (4, '23675543', '9', '2024-10-13 14:00:00', 'Traumatología', 'Cancelado', 'Dolor de rodilla', 'Presencial', 'Ninguna'),
+ (5, '41235894', '5', '2024-10-14 15:30:00', 'Dermatología', 'Confirmado', 'Mancha en la piel', 'Presencial', 'Usar protector');
```

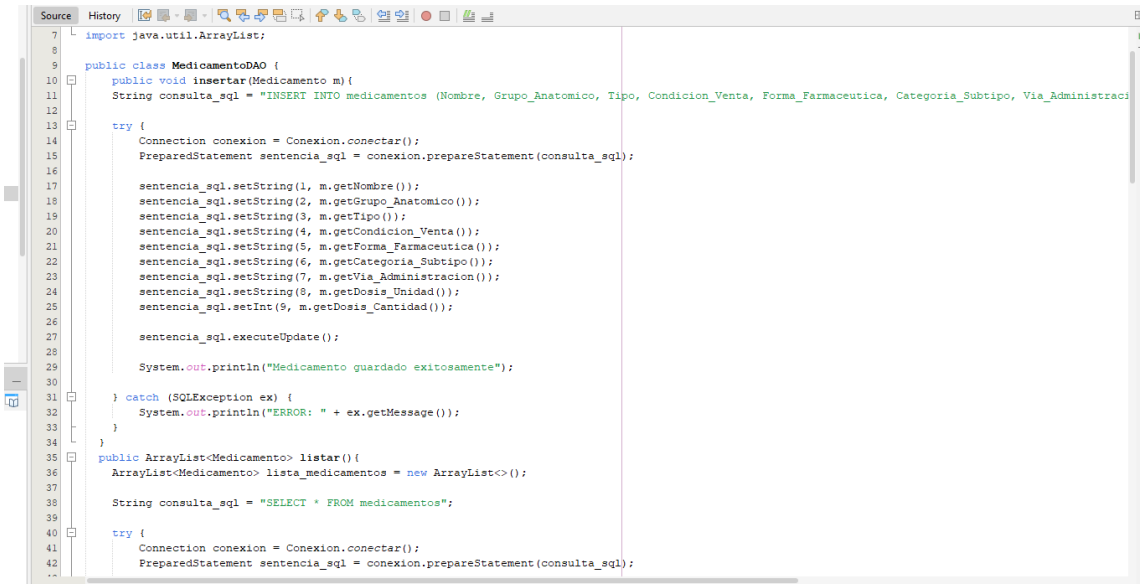
Como no le encontraba sentido a la tabla turnos empecé por ella, modifiqué

Agregué codigo_paciente, nombreCompletoMedico, matricula_medico, estado, motivoConsulta, tipoTurno y una foreign key con médicos por su matrícula. Lo pensé de la siguiente manera: No encontraba mucho sentido que haya tantos ids, como el de medico y paciente, preferí modificar esos datos por codigo_paciente, que se use como dni, Matricula en vez d id, ya que en lo personal creo que lo mejor es que antes de ingresar un turno, tiene que poner la matricula, y esa matricula verifica que si existe un

médico con eso en la base de datos (por eso es unique, o sea única” si no existe, no te deja agregar un turno, creo que es lo mejor para no tener errores a la hora de agregar un turno, como por ejemplo que agregues un turno y el medico que pusiste no existe.

En medicamento, no cambie realmente mucho, solo pase la dosis a int que estaba en varchar.

Después de esto empecé con la creación de los DAO



```
import java.util.ArrayList;

public class MedicamentoDAO {
    public void insertar(Medicamento m){
    String consulta_sql = "INSERT INTO medicamentos (Nombre, Grupo_Anatomico, Tipo, Condicion_Venta, Forma_Farmaceutica, Categoria_Subtipo, Via_Administraci

    try {
        Connection conexion = Conexion.conectar();
        PreparedStatement sentencia_sql = conexion.prepareStatement(consulta_sql);

        sentencia_sql.setString(1, m.getNombre());
        sentencia_sql.setString(2, m.getGrupo_Anatomico());
        sentencia_sql.setString(3, m.getTipo());
        sentencia_sql.setString(4, m.getCondicion_Venta());
        sentencia_sql.setString(5, m.getForma_Farmaceutica());
        sentencia_sql.setString(6, m.getCategoria_Subtipo());
        sentencia_sql.setString(7, m.getVia_Administracion());
        sentencia_sql.setString(8, m.getDosis_Unidad());
        sentencia_sql.setInt(9, m.getDosis_Cantidad());

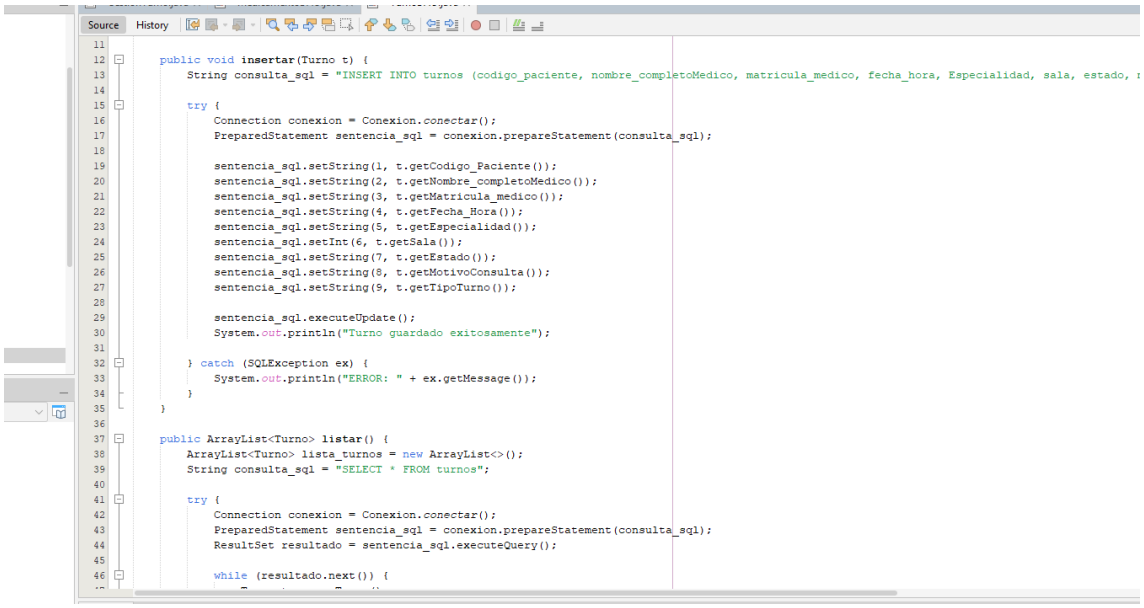
        sentencia_sql.executeUpdate();

        System.out.println("Medicamento guardado exitosamente");

    } catch (SQLException ex) {
        System.out.println("ERROR: " + ex.getMessage());
    }
  }
  public ArrayList<Medicamento> listar(){
    ArrayList<Medicamento> lista_medicamentos = new ArrayList<>();

    String consulta_sql = "SELECT * FROM medicamentos";

    try {
        Connection conexion = Conexion.conectar();
        PreparedStatement sentencia_sql = conexion.prepareStatement(consulta_sql);
```



```
    public void insertar(Turno t) {
        String consulta_sql = "INSERT INTO turnos (codigo_paciente, nombre_completoMedico, matricula_medico, fecha_hora, Especialidad, sala, estado, r

        try {
            Connection conexion = Conexion.conectar();
            PreparedStatement sentencia_sql = conexion.prepareStatement(consulta_sql);

            sentencia_sql.setString(1, t.getCodigo_Paciente());
            sentencia_sql.setString(2, t.getNombre_completoMedico());
            sentencia_sql.setString(3, t.getMatricula_medico());
            sentencia_sql.setString(4, t.getFecha_Hora());
            sentencia_sql.setString(5, t.getEspecialidad());
            sentencia_sql.setInt(6, t.getSala());
            sentencia_sql.setString(7, t.getEstado());
            sentencia_sql.setString(8, t.getMotivoConsulta());
            sentencia_sql.setString(9, t.getTipoTurno());

            sentencia_sql.executeUpdate();
            System.out.println("Turno guardado exitosamente");

        } catch (SQLException ex) {
            System.out.println("ERROR: " + ex.getMessage());
        }
    }

    public ArrayList<Turno> listar() {
        ArrayList<Turno> lista_turnos = new ArrayList<>();
        String consulta_sql = "SELECT * FROM turnos";

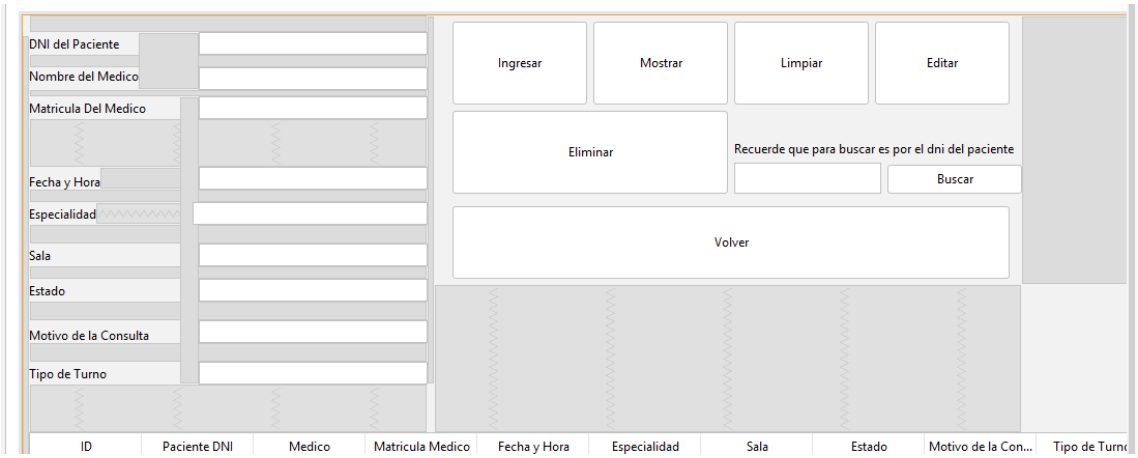
        try {
            Connection conexion = Conexion.conectar();
            PreparedStatement sentencia_sql = conexion.prepareStatement(consulta_sql);
            ResultSet resultado = sentencia_sql.executeQuery();

            while (resultado.next()) {
```

En la clase DAO, realicé nuevas funciones para que se agregue a la base de datos. La conexión la realizo mi compañero Santiago Cañetes, así que me

toco hacer las nuevas funciones de insertar, listar, eliminar, actualizar y buscar.

Después de terminar las funciones pasé a la estética del Jframe, tuve que actualizar el jframe para la función de buscar, ahora busca a través de un txt y quedo así.





Por último, me toca pasar las funciones anteriormente mencionadas a los botones, ya que las funciones que usábamos antes venían desde la clase controladora, ahora vienen del DAO, que está ubicada con la conexión de base de datos. Al final quedo así todo y finalice el día.

Medicamento FUNCIONES

```
    private void btnIngresarActionPerformed(java.awt.event.ActionEvent evt) {
        String nombre = txtNombre.getText();
        String grupoAnatomico = txtGrupoAnatomico.getText();
        String tipo = txtTipo.getText();
        String condicionVenta = txtCondicionVenta.getText();
        String formaFarmaceutica = txtFormaFarmaceutica.getText();
        String categoriaSubtipo = txtCategoriaSubtipo.getText();
        String viaAdministracion = txtViaAdministracion.getText();
        String dosisUnidad = txtDosisUnidad.getText();
        int dosisCantidad = Integer.parseInt(txtDosisCantidad.getText());

        Medicamento nuevoMedicamento = new Medicamento( nombre,  grupoAnatomico,  tipo,  condicionVenta,  formaFarmaceutica,  categoriaSubtipo,  viaAdmini

        //CP.AgregarMedicamento(nuevoMedicamento);

        MedicamentoDAO dao = new MedicamentoDAO();

        dao.insertar(nuevoMedicamento);


        JOptionPane.showMessageDialog(null, "¡Medicamento registrado exitosamente!");

        txtNombre.setText("");
        txtGrupoAnatomico.setText("");
        txtTipo.setText("");
        txtCondicionVenta.setText("");
        txtFormaFarmaceutica.setText("");
        txtCategoriaSubtipo.setText("");
        txtViaAdministracion.setText("");
        txtDosisUnidad.setText("");
        txtDosisCantidad.setText("");

    }

    public void mostrarMedicamentoEnTabla(Medicamento M){
        DefaultTableModel modelo = (DefaultTableModel)tablaMedicamentos.getModel();
```

Turnos FUNCIONES

```
urce  Design  History
    private void IngresarActionPerformed(java.awt.event.ActionEvent evt) {

        String codigoPaciente = txtCodigoPaciente.getText();
        String nombreMedico = txtNombreMedico.getText();
        String matriculaMedico = txtMatriculaMedico.getText();
        String fechaHora = txtFecha_Hora.getText();
        String especialidad = txtEspecialidad.getText();
        int sala = Integer.parseInt(txtSala.getText());
        String estado = txtEstado.getText();
        String motivoConsulta = txtMotivoConsulta.getText();
        String tipoTurno = txtTipoTurno.getText();

        Turno nuevoTurno = new Turno(codigoPaciente, nombreMedico, matriculaMedico, fechaHora, especialidad, sala, estado, motivoConsulta, tipoTur

        TurnoDAO dao = new TurnoDAO();
        dao.insertar(nuevoTurno);

        JOptionPane.showMessageDialog(null, "Turno registrado exitosamente");


        txtCodigoPaciente.setText("");
        txtNombreMedico.setText("");
        txtMatriculaMedico.setText("");
        txtFecha_Hora.setText("");
        txtEspecialidad.setText("");
        txtSala.setText("");
        txtEstado.setText("");
        txtMotivoConsulta.setText("");
        txtTipoTurno.setText("");
        }

        private void btnMostrarActionPerformed(java.awt.event.ActionEvent evt) {

        TurnoDAO dao = new TurnoDAO();
        ArrayList<Turno> lista_turnos = dao.listar();
        DefaultTableModel modelo = (DefaultTableModel) tablaTurno.getModel();
```

Además, modifique en la base de datos los insert, ya que habían algunos que no funcionaban de otras tablas y tenía que agregar los nuevos datos que agregué



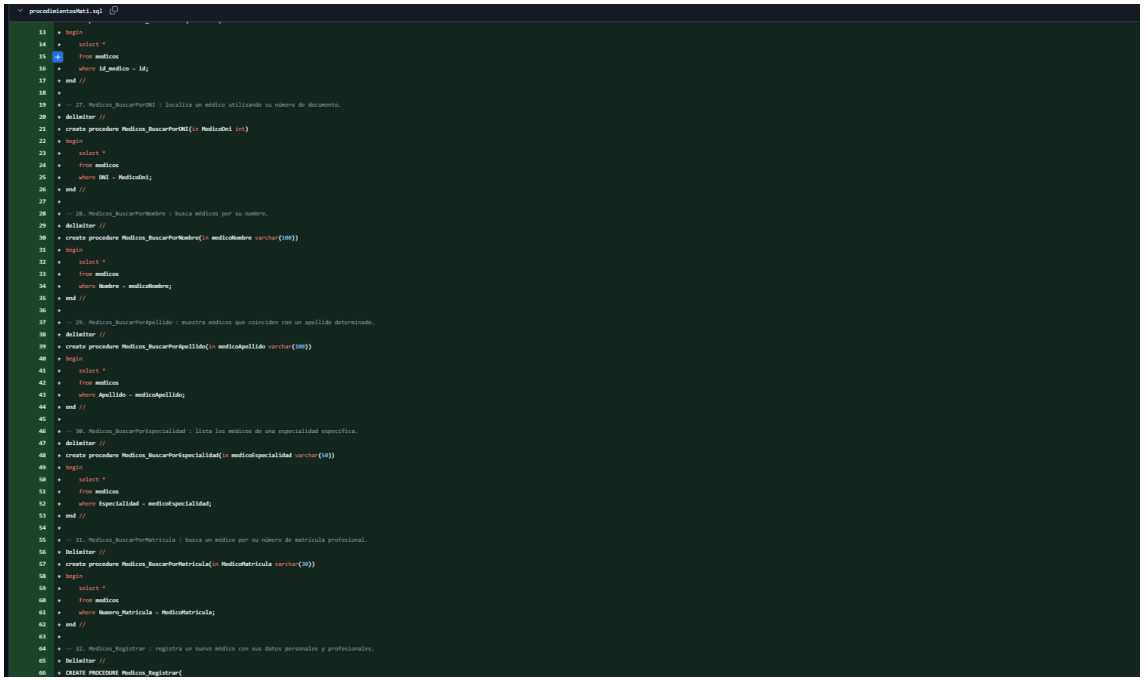
20/10/26

Lugar: Laboratorio

Hora: 9:55

Actividad

Hoy me puse hacer los 120 procedimientos que me había asignado mi líder, como sé procedimientos y me va dentro de todo bien, pude hacerlos sin ningún problema, solo que algunos no estaban bien, asi que tuve que modificar nuevamente los INSERT, para que funcionase bien



Y así con los 120 procedimientos hechos finalice el día

21/10/26

Lugar: Laboratorio

Hora: 9:55

Actividad:

Este día no se realizó actividades

22/10/26

Lugar: Laboratorio

Hora: 9:55

Actividad:

23/10/26

Lugar: Laboratorio

Hora: 9:55

Actividad:

El día de hoy realice

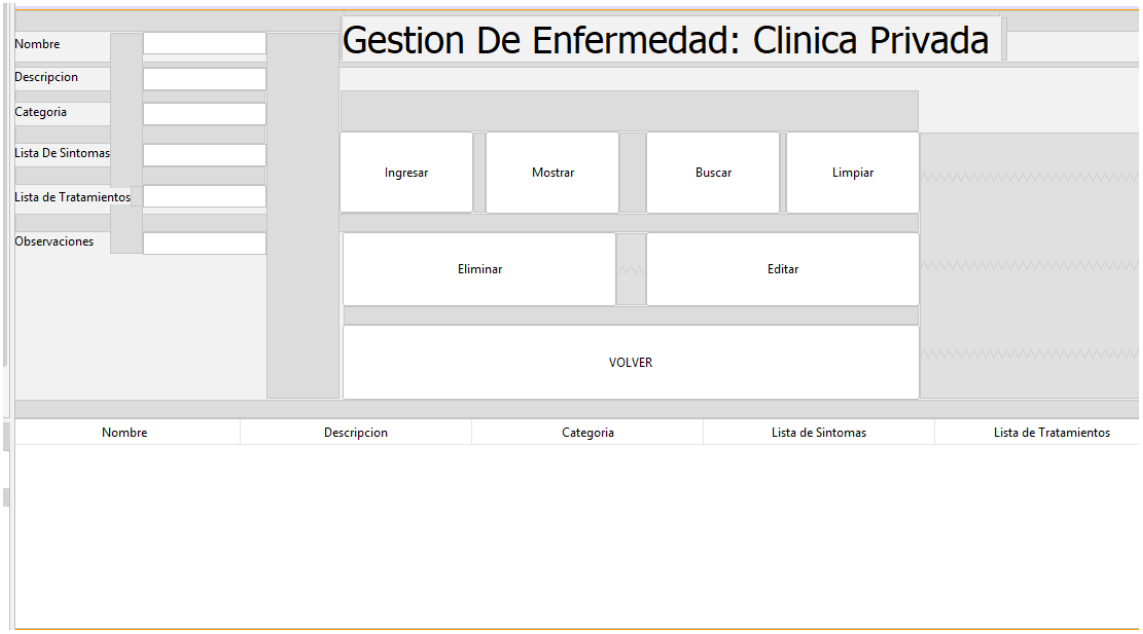
23/10/26

Lugar: Laboratorio

Hora: 9:55

Actividad:

El día de hoy seguí con POO, ya que mi compañero Luciano Villarroel y Santiago Cañete hicieron sus clases debidas en distintos archivos, tuve que ponerme a trabajar y juntarlos así solo había 1 archivo. Agarré el archivo que había usado Luciano y plasmé lo de Enfermedad desde ahí.



Así se veía antes de la modificación, le faltaba aplicar lo del DAO



Así quedo finalmente una vez terminado el Jframe para empezar con sus funcionamientos con el DAO. Empecé con las funcionalidades, solo era copiar y pegar, pero antes de eso verifiqué si había algún problema o error, pero no, estaba todo bien así que sigue pasándolo con normalidad

```
private void btnMostrarActionPerformed(java.awt.event.ActionEvent evt) {
 EnfermedadDAO dao = new EnfermedadDAO();
ArrayList<Enfermedad> listaEnfermedades = dao.listarEnfermedad();

DefaultTableModel modelo = (DefaultTableModel) tablaEnfermedad.getModel();
modelo.setRowCount(0);

for (Enfermedad enfermedad : listaEnfermedades) {
    modelo.addRow(new Object[]{
        enfermedad.getIdEnfermedad(),
        enfermedad.getNombre(),
        enfermedad.getDescripcion(),
        enfermedad.getCategoria(),
        enfermedad.getLista_Sintomas(),
        enfermedad.getLista_Tratamientos(),
        enfermedad.getObservaciones()
    });
    }
```

```
private void btnIngresarActionPerformed(java.awt.event.ActionEvent evt) {
    String Nombre = txtNombre.getText();
    String Descripcion = txtDescripcion.getText();
    String Categoria = txtCategoria.getText();
    String Lista_sintomas = txtListaSintomas.getText();
    String Lista_tratamientos = txtListaTratamientos.getText();
    String Observaciones = txtObservaciones.getText();

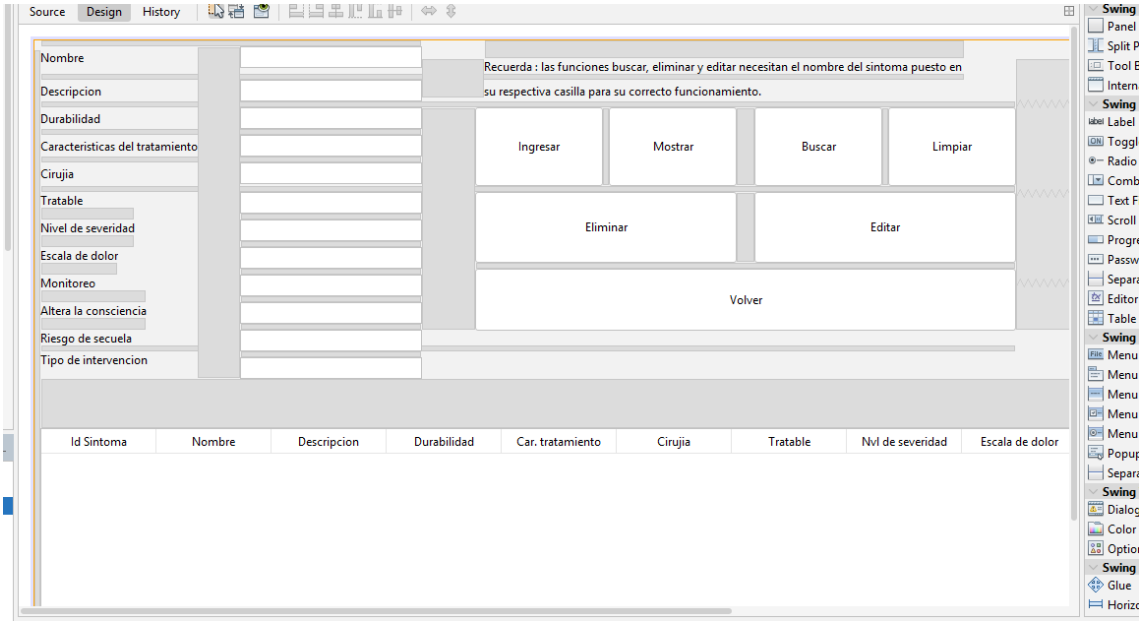
    Enfermedad nuevaEnfermedad = new Enfermedad( Nombre, Descripcion, Categoria, Lista_sintomas, Lista_tratamientos, Observaciones);

    EnfermedadDAO dao = new EnfermedadDAO();

    dao.insertarEnfermedad(nuevaEnfermedad);

    JOptionPane.showMessageDialog(null, "Enfermedad registrada exitosamente !");
    txtNombre.setText("");
    txtDescripcion.setText("");
    txtCategoria.setText("");
    txtListaSintomas.setText("");
    txtListaTratamientos.setText("");
    txtObservaciones.setText("");
}
```

Una vez terminé de pasar todo a blanco en un solo archivo, antes de cerrarlo me acordé que en sintoma faltaba agregar el mouseClicked, la función de buscar y el txt para buscar en el JFrame, asi que lo agregué pasando esto



A esto



Además, agregándole en los botones la función correspondida del MouseClicked y el buscar (función hecha anteriormente por Villarroel en el DAO)

```
    private void btnBuscarActionPerformed(java.awt.event.ActionEvent evt) {
        String nombreBuscado = txtNombreaBuscar.getText();
        SintomaDAO dao = new SintomaDAO();

    ArrayList<Sintoma> sintomas = dao.buscarSintoma(nombreBuscado);

    DefaultTableModel modelo = (DefaultTableModel) tablaSintomas.getModel();

    modelo.setRowCount(0);

    for (Sintoma sin : sintomas) {
        modelo.addRow(new Object[]{

            sin.getId_sintoma(),
            sin.getNombre(),
            sin.getDescripcion(),
            sin.getDurabilidad(),
            sin.getCaracteristicas_Tratamiento(),
            sin.getCirujia(),
            sin.getTratable(),
            sin.getNivel_Severidad(),
            sin.getEscala_Dolor(),
            sin.getRequiere_Monitoreo(),
            sin.getAltera_Consciencia(),
            sin.getRiesgo_Secuela(),
            sin.getTipo_Intervencion()
        });
    }
    }
```

```
private void tablaSintomasMouseClicked(java.awt.event.MouseEvent evt) {
int fila = tablaSintomas.getSelectedRow();

if (fila == -1) {
    return;
}

txtNombre.setText(tablaSintomas.getValueAt(fila, 1).toString());
txtDescripcion.setText(tablaSintomas.getValueAt(fila, 2).toString());
txtDurabilidad.setText(tablaSintomas.getValueAt(fila, 3).toString());
txtCTratamiento.setText(tablaSintomas.getValueAt(fila, 4).toString());
txtCirujia.setText(tablaSintomas.getValueAt(fila, 5).toString());
txtTratable.setText(tablaSintomas.getValueAt(fila, 6).toString());
txtNvlSeveridad.setText(tablaSintomas.getValueAt(fila, 7).toString());
txtEscDolor.setText(tablaSintomas.getValueAt(fila, 8).toString());
txtMonitoreo.setText(tablaSintomas.getValueAt(fila, 9).toString());
txtAlteraConsciencia.setText(tablaSintomas.getValueAt(fila, 10).toString());
txtRiesgo.setText(tablaSintomas.getValueAt(fila, 11).toString());
txtTipoIntervencion.setText(tablaSintomas.getValueAt(fila, 12).toString());
```

Así mismo, finalizando el día realizando esto.

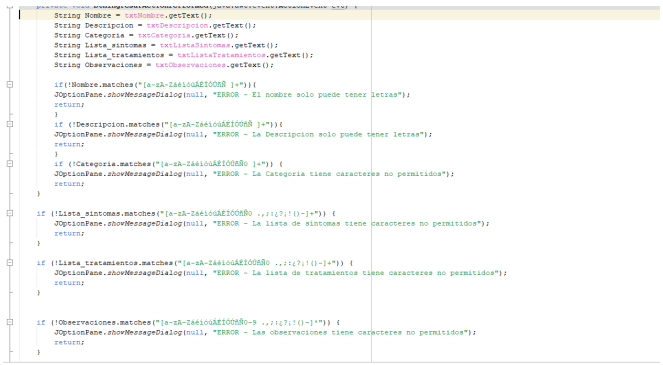
25/10/26

Lugar: Laboratorio

Hora: 9:55

Actividad:

Hoy continúe con mis tareas, me fijé en el Gantt y me faltaba realizar validaciones, entonces me puse con eso. Entré al GitHub y resulta que mi compañera Martina Orlandi había realizado las validaciones de Turno, así que tuve una tabla menos por realizar, hice todas las tablas restantes. Como anteriormente nuestro profesor Nahuel Acosta nos había enseñado, no me resulto difícil, lo hice fácilmente.



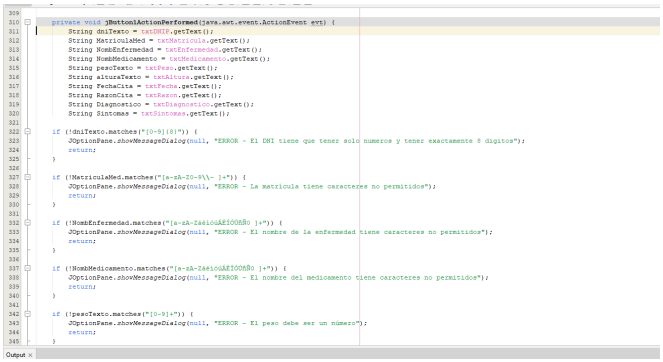
```
        String Nombre = txtNombre.getText();
        String Descripcion = txtDescripcion.getText();
        String Categoria = txtCategoria.getText();
        String Lista_sintomas = txtListaSintomas.getText();
        String Lista_tratamientos = txtListaTratamientos.getText();
        String Observaciones = txtObservaciones.getText();

        if(!Nombre.matches("[a-zA-ZáéíóúÁÉÍÓÚÑ ]+")){
        JOptionPane.showMessageDialog(null, "ERROR - El nombre solo puede tener letras");
        return;
        }
        if (!Descripcion.matches("[a-zA-ZáéíóúÁÉÍÓÚÑ ]+")){
        JOptionPane.showMessageDialog(null, "ERROR - La Descripcion solo puede tener letras");
        return;
        }
        if (!Categoria.matches("[a-zA-ZáéíóúÁÉÍÓÚÑ ]+")) {
        JOptionPane.showMessageDialog(null, "ERROR - La Categoria tiene caracteres no permitidos");
        return;
    }

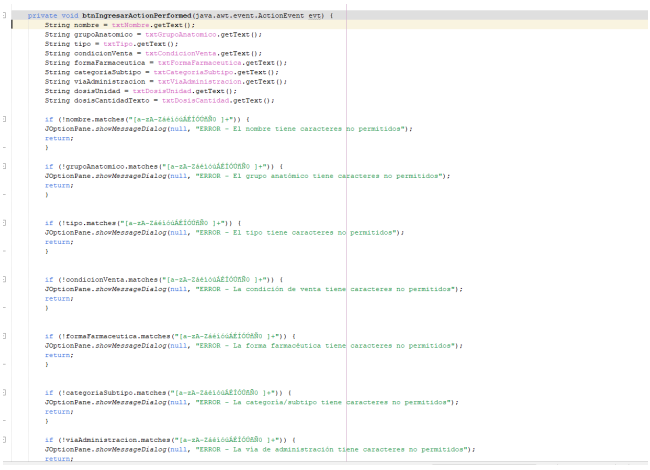
    if (!Lista_sintomas.matches("[a-zA-ZáéíóúÁÉÍÓÚÑ .,;:¿?¡!()-]+")) {
        JOptionPane.showMessageDialog(null, "ERROR - La lista de sintomas tiene caracteres no permitidos");
        return;
    }

    if (!Lista_tratamientos.matches("[a-zA-ZáéíóúÁÉÍÓÚÑ .,;:¿?¡!()-]+")) {
        JOptionPane.showMessageDialog(null, "ERROR - La lista de tratamientos tiene caracteres no permitidos");
        return;
    }


    if (!Observaciones.matches("[a-zA-ZáéíóúÁÉÍÓÚÑ0-9 .,;:¿?¡!()-]*")) {
        JOptionPane.showMessageDialog(null, "ERROR - Las observaciones tiene caracteres no permitidos");
        return;
    }
```



```
309
310     private void jButton1ActionPerformed(java.awt.event.ActionEvent evt) {
311         String dniTexto = txtDNIP.getText();
312         String MatriculaMed = txtMatricula.getText();
313         String NombEnfermedad = txtEnfermedad.getText();
314         String NombMedicamento = txtMedicamento.getText();
315         String pesoTexto = txtPeso.getText();
316         String alturaTexto = txtAltura.getText();
317         String FechaCita = txtFecha.getText();
318         String RazonCita = txtRazon.getText();
319         String Diagnostico = txtDiagnostico.getText();
320         String Sintomas = txtSintomas.getText();
321
322         if (!dniTexto.matches("[0-9]{8}")) {
323             JOptionPane.showMessageDialog(null, "ERROR - El DNI tiene que tener solo numeros y tener exactamente 8 digitos");
324             return;
325         }
326
327         if (!MatriculaMed.matches("[a-zA-Z0-9\\- ]+")) {
328             JOptionPane.showMessageDialog(null, "ERROR - La matricula tiene caracteres no permitidos");
329             return;
330         }
331
332         if (!NombEnfermedad.matches("[a-zA-ZáéíóúÁÉÍÓÚÑ ]+")) {
333             JOptionPane.showMessageDialog(null, "ERROR - El nombre de la enfermedad tiene caracteres no permitidos");
334             return;
335         }
336
337         if (!NombMedicamento.matches("[a-zA-ZáéíóúÁÉÍÓÚÑ ]+")) {
338             JOptionPane.showMessageDialog(null, "ERROR - El nombre del medicamento tiene caracteres no permitidos");
339             return;
340         }
341
342         if (!pesoTexto.matches("[0-9]+")) {
343             JOptionPane.showMessageDialog(null, "ERROR - El peso debe ser un número");
344             return;
345         }
Output
```



```
    private void btnIngresarActionPerformed(java.awt.event.ActionEvent evt) {
        String nombre = txtNombre.getText();
        String grupoAnatomico = txtGrupoAnatomico.getText();
        String tipo = txtTipo.getText();
        String condicionVenta = txtCondicionVenta.getText();
        String formaFarmaceutica = txtFormaFarmaceutica.getText();
        String categoriaSubtipo = txtCategoriaSubtipo.getText();
        String viaAdministracion = txtViaAdministracion.getText();
        String dosisUnidad = txtDosisUnidad.getText();
        String dosisCantidadTexto = txtDosisCantidad.getText();

        if (!nombre.matches("[a-zA-ZáéíóúÁÉÍÓÚÑ ]+")) {
        JOptionPane.showMessageDialog(null, "ERROR - El nombre tiene caracteres no permitidos");
        return;
        }

        if (!grupoAnatomico.matches("[a-zA-ZáéíóúÁÉÍÓÚÑ ]+")) {
        JOptionPane.showMessageDialog(null, "ERROR - El grupo anatómico tiene caracteres no permitidos");
        return;
        }

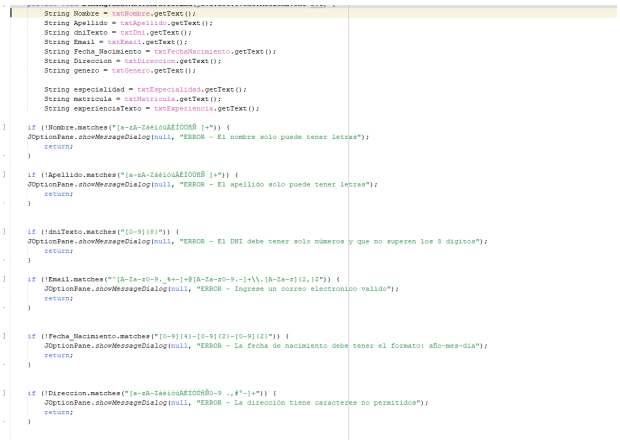

        if (!tipo.matches("[a-zA-ZáéíóúÁÉÍÓÚÑ ]+")) {
        JOptionPane.showMessageDialog(null, "ERROR - El tipo tiene caracteres no permitidos");
        return;
        }


        if (!condicionVenta.matches("[a-zA-ZáéíóúÁÉÍÓÚÑ ]+")) {
        JOptionPane.showMessageDialog(null, "ERROR - La condición de venta tiene caracteres no permitidos");
        return;
        }


        if (!formaFarmaceutica.matches("[a-zA-ZáéíóúÁÉÍÓÚÑ ]+")) {
        JOptionPane.showMessageDialog(null, "ERROR - La forma farmacéutica tiene caracteres no permitidos");
        return;
        }


        if (!categoriaSubtipo.matches("[a-zA-ZáéíóúÁÉÍÓÚÑ ]+")) {
        JOptionPane.showMessageDialog(null, "ERROR - La categoria/subtipo tiene caracteres no permitidos");
        return;
        }

        if (!viaAdministracion.matches("[a-zA-ZáéíóúÁÉÍÓÚÑ ]+")) {
        JOptionPane.showMessageDialog(null, "ERROR - La via de administración tiene caracteres no permitidos");
        return;
```



```
        String Nombre = txtNombre.getText();
        String Apellido = txtApellido.getText();
        String dniTexto = txtDni.getText();
        String Email = txtEmail.getText();
        String Fecha_Nacimiento = txtFechaNacimiento.getText();
        String Direccion = txtDireccion.getText();
        String genero = txtGenero.getText();

        String especialidad = txtEspecialidad.getText();
        String matricula = txtMatricula.getText();
        String experienciaTexto = txtExperiencia.getText();

    if (!Nombre.matches("[a-zA-ZáéíóúÁÉÍÓÚñÑ ]+")) {
    JOptionPane.showMessageDialog(null, "ERROR - El nombre solo puede tener letras");
        return;
    }

    if (!Apellido.matches("[a-zA-ZáéíóúÁÉÍÓÚñÑ ]+")) {
    JOptionPane.showMessageDialog(null, "ERROR - El apellido solo puede tener letras");
        return;
    }

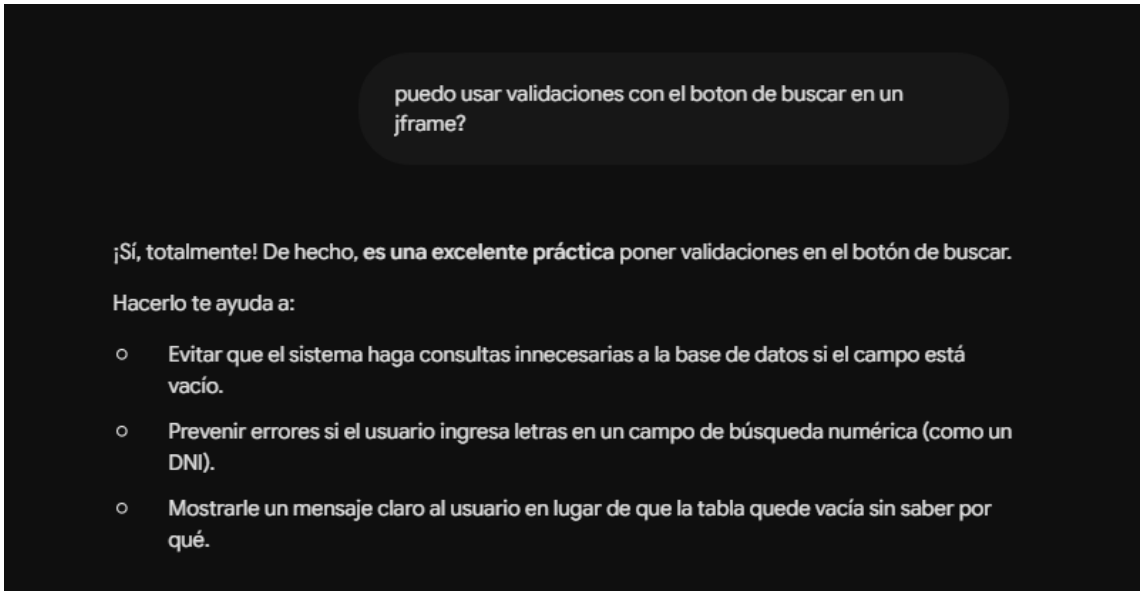

    if (!dniTexto.matches("[0-9]{8}")) {
    JOptionPane.showMessageDialog(null, "ERROR - El DNI debe tener solo números y que no superen los 8 digitos");
        return;
    }

    if (!Email.matches("^[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\\.[A-Za-z]{2,}$")) {
        JOptionPane.showMessageDialog(null, "ERROR - Ingrese un correo electronico valido");
        return;
    }

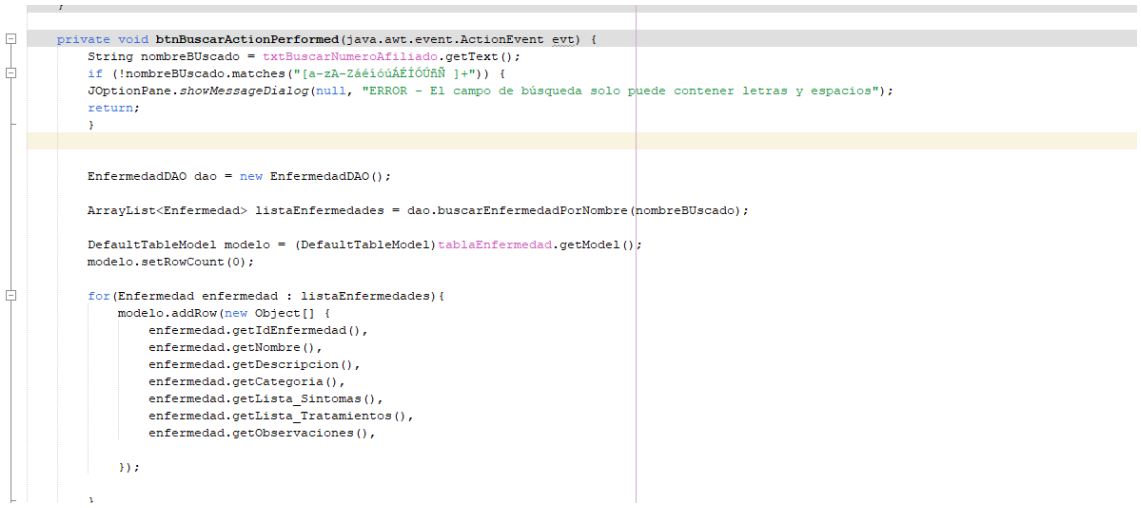

    if (!Fecha_Nacimiento.matches("[0-9]{4}-[0-9]{2}-[0-9]{2}")) {
        JOptionPane.showMessageDialog(null, "ERROR - La fecha de nacimiento debe tener el formato: año-mes-dia");
        return;
    }


    if (!Direccion.matches("[a-zA-ZáéíóúÁÉÍÓÚñÑ0-9 .,#°-]+")) {
        JOptionPane.showMessageDialog(null, "ERROR - La dirección tiene caracteres no permitidos");
        return;
    }
```

Esas son algunas fotos de las validaciones, faltan más, pero esas son algunas. Antes de cerrar el Netbeans, me puse a pensar y para que el programa sea más seguro, quise saber si las validaciones también se podían implementar en el txt y botón de buscar, así que le pregunté a la IA a ver si era posible y me dijo esto



Así que, en base a eso, le pedí que me muestre una estructura de como se hace y me dijo que se hace como las validaciones que venía haciendo en el botón de insertar, así que me puse a completar las validaciones en el botón de buscar



```
}

private void btnBuscarActionPerformed(java.awt.event.ActionEvent evt) {
    String nombreBUscado = txtBuscarNumeroAfiliado.getText();
    if (!nombreBUscado.matches("[a-zA-ZáéíóúÁÉÍÓÚñÑ ]+")) {
    JOptionPane.showMessageDialog(null, "ERROR - El campo de búsqueda solo puede contener letras y espacios");
    return;
    }


    EnfermedadDAO dao = new EnfermedadDAO();

    ArrayList<Enfermedad> listaEnfermedades = dao.buscarEnfermedadPorNombre(nombreBUscado);

    DefaultTableModel modelo = (DefaultTableModel)tablaEnfermedad.getModel();
    modelo.setRowCount(0);

    for(Enfermedad enfermedad : listaEnfermedades){
        modelo.addRow(new Object[] {
            enfermedad.getIdEnfermedad(),
            enfermedad.getNombre(),
            enfermedad.getDescripcion(),
            enfermedad.getCategoria(),
            enfermedad.getLista_Sintomas(),
            enfermedad.getLista_Tratamientos(),
            enfermedad.getObservaciones(),

        });
    }
```

Y así lo hice en todas las tablas, una vez terminé lo subí al GitHub, así terminando el día.

26/10/26

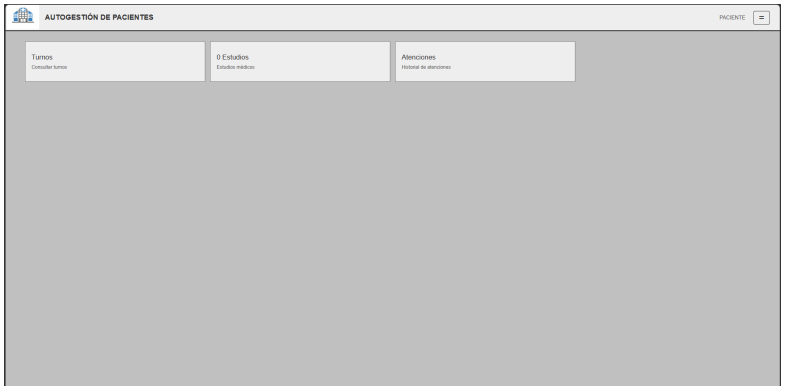
Lugar: Laboratorio

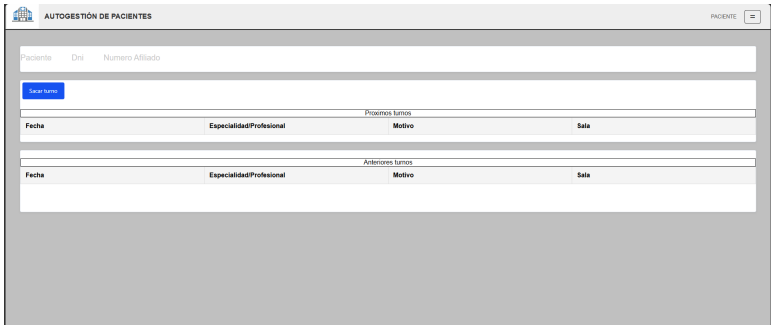
Hora: 9:55

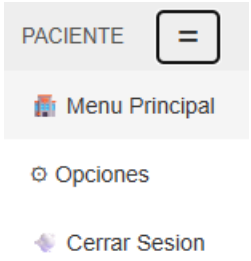
Actividad:

Hoy fue la 3ra entrega del proyecto, apenas llegamos nuevamente los profesores nos pidieron que coloquemos las cosas en las compus. Lamentablemente, los profesores no llegaron a llamarnos, así que nos dijeron que el miércoles llamarían a los grupos que faltaban. Así que seguí con la página web. Lo que hice que en la pantalla de gestión del paciente reemplacé el texto de la barra superior por el logo en una img. Agregué un menú desplegable con un button y una lista (ul y li con a) con “Menú principal”, “Opciones” y “Cerrar sesión”. Con JavaScript lo abro y cierro al hacer clic en el botón y también lo cierro al hacer clic afuera, usando addEventListener y classList.toggle. Las tres opciones (Turnos, Estudios y Atenciones) ahora son clickeables, cada una usa oneclick y window.location.href para llevar a su página. En la página de turnos usé la misma barra y el mismo menú, y agregué un bloque con los datos del paciente (h2 con nombre, DNI y número de afiliado), un botón “Sacar turno” el cual te redirecciona a una página que todavía hay que modificar. Para los próximos turnos y los turnos anteriores, use table, thead, tr, th, con las columnas fecha, especialidad/profesional, motivo y sala. Las

páginas de Atenciones y Estudios las dejé creadas con la estructura básica de HTML para seguirlas otro día. Finalmente, después de todos los cambios quedó así







Así terminando el día de hoy

27/10/26

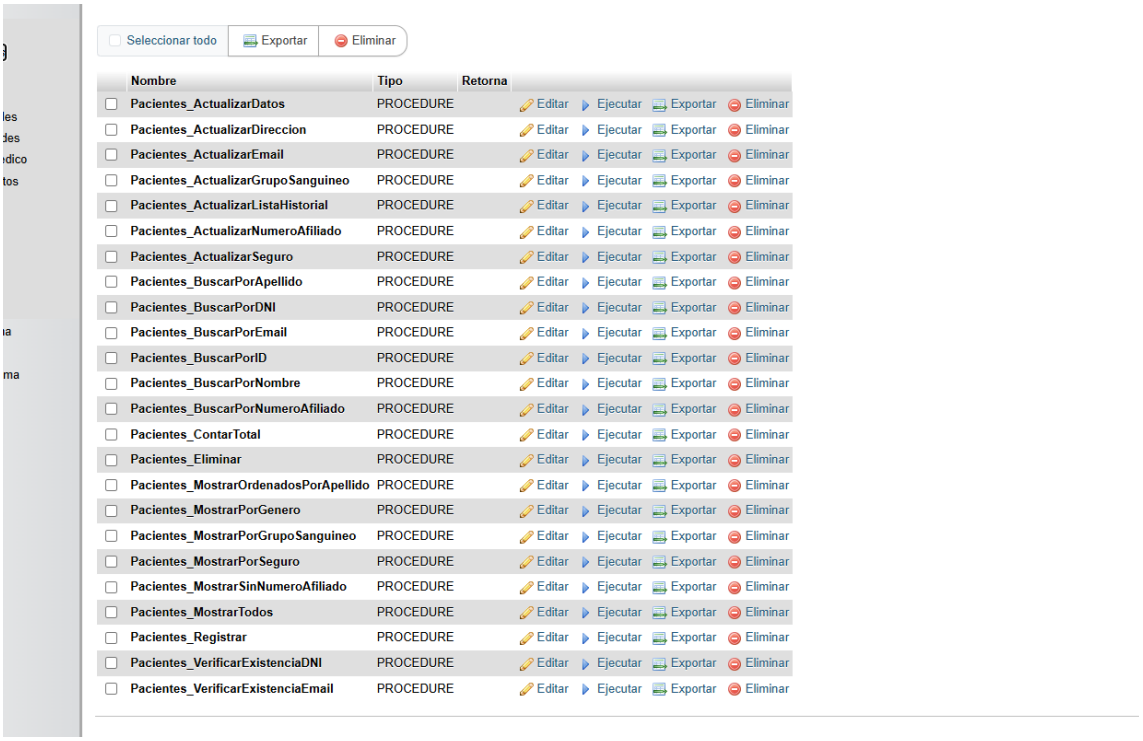
Lugar: Laboratorio

Hora: 9:55

Actividad:

Hoy antes de la entrega del día de mañana decidí cambiar los INSERT que usábamos a los procedimientos que habíamos realizado previamente, me puse manos a la obra, pero antes de empezar, me di cuenta que no sabía cómo hacerlo, así que le pregunté al profesor Nahuel Acosta.

Lamentablemente como tenía muchos llamados no pudo ayudarme, así que Benjamín Saragusti, se ofreció a explicarme como era. Me explico que primero tengo que pasar los procedimientos a phpMyAdmin,



una vez los coloqué, tengo que abrir NetBeans y hacer estos cambios, primero modificar el anteriormente insert en la clase DAO a esto

```
"call Pacientes_Registrar(?,?,?,?,?,?,?,?,?,?,?)";
```

Después cambiar Stament y usar el callBle

```
CallableStatement sentencia_sql = conexion.prepareCall(consulta_sql);
```

Y solo así se hace usa los procedimientos en POO, así que me puse a realizar los restantes.

```
public void insertar(Medicamento m) {
    String consulta_sql = "call Medicamentos_Registrar(?, ?, ?, ?, ?, ?, ?, ?)";

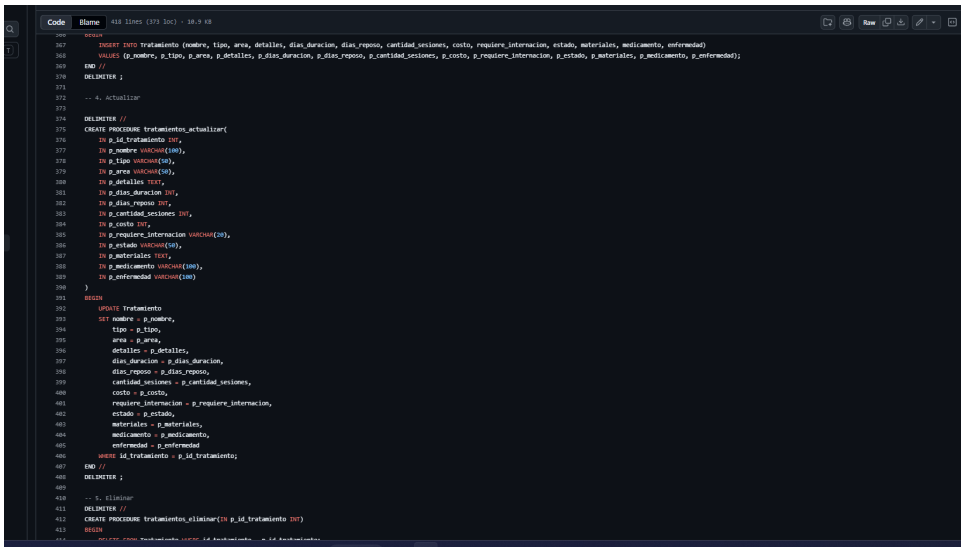
    try {
        Connection conexion = Conexion.conectar();
        CallableStatement sentencia_sql = conexion.prepareCall(consulta_sql);

        sentencia_sql.setString(1, m.getNombre());
        sentencia_sql.setString(2, m.getGrupo_Anatomico());
        sentencia_sql.setString(3, m.getTipo());
        sentencia_sql.setString(4, m.getCondicion_Venta());
        sentencia_sql.setString(5, m.getForma_Farmaceutica());
        sentencia_sql.setString(6, m.getCategoria_Subtipo());
        sentencia_sql.setString(7, m.getVia_Administracion());
        sentencia_sql.setString(8, m.getDosis_Unidad());
        sentencia_sql.setInt(9, m.getDosis_Cantidad());

        sentencia_sql.executeUpdate();
        System.out.println("Medicamento guardado exitosamente");

    } catch (SQLException ex) {
        System.out.println("ERROR: " + ex.getMessage());
    }
}
```

Aunque cuando los estaba realizando, tuve algunos errores con procedimientos que hicieron, o que faltaban algunos procedimientos asi que los agregué. Como la tabla de tratamientos es nueva, no se realizaron los procedimientos, así que para terminar con todos los store procedure en java tuve que hacerlos



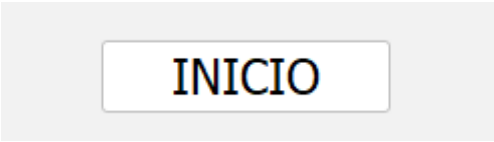
```
Code  Blame  418 lines (373 loc) · 10.9 KB                                                
```

cosas que faltaban, como la opción para que el paciente pueda editar sus datos, la conexión con JAVA y MySQL, además decorar todo el HTML porque estaba feo.

Después de la entrevista, hable con mi compañero Santiago Cañetes para ponernos de acuerdo de lo que faltaba para la entrega final del 6 de noviembre, cree la parte del GANTT de la entrega final, y el líder asigno todas las tareas restantes

	A	B	C	D	E	F	G	H	I	J	K
1	**POO**										
2	Decoracion de Aplicacion (JFrame)	Martina Orlandi			Pendiente	1	2	3	4	5	6
3	Conexion Java - Web - SQL	Luciano Villarroel, Martina Orlandi			Pendiente						
4											
5	**Proyecto Informatico 2.**										
6	Registrar Turnos En la Base De Datos Desde La Pagina.	Santiago Cañete			Pendiente						
7	Completar Pagina De Estudios	Martina Orlandi			Pendiente						
8	Completar Pagina Historial Medico	Bautista Miyashiro			Pendiente						
9	Pagina Modificacion de Datos.	Matias Basto, Luciano Villarroel			Pendiente						
10	CSS de todo lo Anterior	Matias Basto			Pendiente						
11	**Redes.**										
12	Configurar 39 Compus	Martina Orlandi			Pendiente						
13	***Subir El Trabajo Hecho al Github***										
14	**AGBD**										
15	Hacer Triggers, 10 Paciente, 10 Especialidad	Santiago Cañete			Pendiente						
16	HAcer Triggers, 10 Medicos, 10 Turnos	Matias Basto			Pendiente						
17	Hacer Triggers, 10 MEdicamentos, 10 enfermedades.	Bautista Miyashiro			Pendiente						
18	Hacer Triggers, 10 Historial, 10 Usuarios	Luciano Villarroel			Pendiente						
19	Hacer Triggers, 10 Sintoma , 10 Tratamiento	Martina Orlandi			Pendiente						
20											
21	Hacer Vistas, 10 Paciente, 10 Especialidad	Santiago Cañete			Pendiente						
22	HAcer Vistas, 10 Medicos, 10 Turnos	Matias Basto			Pendiente						
23	Hacer Vistas, 10 Medicamentos, 10 enfermedades	Bautista Miyashiro			Pendiente						
24	Hacer Vistas, 10 Historial, 10 Usuarios	Luciano Villarroel			Pendiente						
25	Hacer Vistas, 10 Sintoma , 10 Tratamiento	Martina Orlandi			Pendiente						
26	**TESTEO**										
27	Testear La Aplicacion de Java, Pulirla y Subirla.				Pendiente						
28	Testear Y Unir la Base de Datos Pulirla y Subirla.				Pendiente						
29	Testear La Pagina web Pulirla y Subirla.				Pendiente						
30	Testear el Cisco packet Tracer, Pulirlo y Subirlo.				Pendiente						
31	Testear				Pendiente						
32					Pendiente						
33											
34											
35											
36											
37											
38	**MAQUETA : Minecraft.**										

En este mismo día me puse a trabajar sobre POO, agregué el botón para volver al inicio desde la pantalla de selección y verifiqué que funcionara.



Así finalizando el día

29/10/26

Lugar: Aula Informática

Hora: 7:40

Actividad:

30/10/26

Lugar: Aula Informática

Hora: 7:40

Actividad:

Hoy seguí con lo nos quedó pendiente para la entrega final, me anoté lo que me comento los profesores así que me puse a darle uso a la tabla de especialidades de POO, primero creando la clase, con 3 atributos, id, nombre y descripción,



```java
package clinica_privada_poo_2026;

public class Especialidades {
    private int id_especialidad;
    private String nombre;
    private String descripcion;

    public Especialidades(String nombre, String descripcion) {
        this.nombre = nombre;
        this.descripcion = descripcion;

    }

    public int getId_especialidad() {
        return id_especialidad;
    }

    public void setId_especialidad(int id_especialidad) {
        this.id_especialidad = id_especialidad;
    }

    public String getNombre() {
        return nombre;
    }

    public void setNombre(String nombre) {
        this.nombre = nombre;
    }

    public String getDescripcion() {
        return descripcion;
    }

    public void setDescripcion(String descripcion) {
        this.descripcion = descripcion;
    }
```

después hice el DAO con sus funciones, insertar, listar, modificar, eliminar y buscar, usando los procedimientos que como era una nueva tabla los tuve que hacer

```sql
-- 1. mostrar todos
delimiter //
create procedure especialidades_mostrartodos()
begin
    select * from especialidades;
end //
delimiter ;

-- 2. buscar por nombre
delimiter //
create procedure especialidades_buscarpornombre(in p_nombre varchar(50))
begin
    select * from especialidades where nombre = p_nombre;
end //
delimiter ;

-- 3. registrar
delimiter //
create procedure especialidades_registrar(
    in p_nombre varchar(50),
    in p_descripcion text
)
begin
    insert into especialidades (nombre, descripcion)
    values (p_nombre, p_descripcion);
end //
delimiter ;

-- 4. actualizar
delimiter //
create procedure especialidades_actualizar(
    in p_id_especialidad int,
    in p_nombre varchar(50),
    in p_descripcion text
)
begin
    update especialidades
    set nombre = p_nombre,
        descripcion = p_descripcion
    where id_especialidad = p_id_especialidad;
end //
delimiter ;

-- 5. eliminar
delimiter //
create procedure especialidades_eliminar(in p_id_especialidad int)
begin
```

```java
import java.sql.CallableStatement;

public class EspecialidadesDAO {

    public void insertar(Especialidades e) {
        String consulta_sql = "call especialidades_registrar(?, ?)";

        try {
            Connection conexion = Conexion.conectar();
            CallableStatement sentencia_sql = conexion.prepareCall(consulta_sql);

            sentencia_sql.setString(1, e.getNombre());
            sentencia_sql.setString(2, e.getDescripcion());

            sentencia_sql.executeUpdate();
            System.out.println("Especialidad guardada exitosamente");

        } catch (SQLException ex) {
            System.out.println("ERROR: " + ex.getMessage());
        }
    }

    public ArrayList<Especialidades> listar() {
            ArrayList<Especialidades> lista_e = new ArrayList<>();
            String consulta_sql = "call especialidades_mostrartodos()";

            try {
                Connection conexion = Conexion.conectar();
                CallableStatement sentencia_sql = conexion.prepareCall(consulta_sql);

                ResultSet resultado = sentencia_sql.executeQuery();

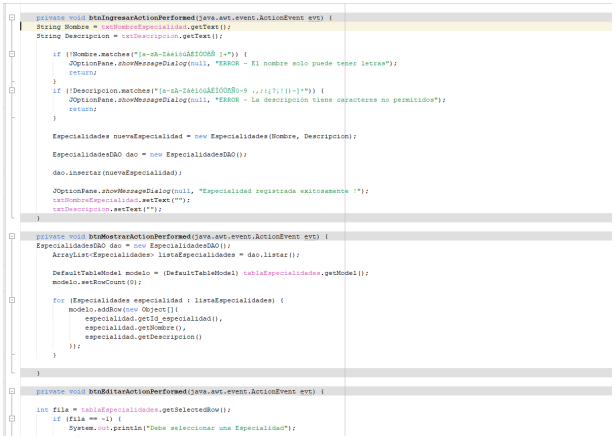
                while(resultado.next()) {
                    Especialidades e = new Especialidades();

                    e.setId_especialidad(resultado.getInt("id_especialidad"));
                    e.setNombre(resultado.getString("nombre"));
                    e.setDescripcion(resultado.getString("descripcion"));

                    lista_e.add(e);
                }
            } catch (SQLException ex) {
```

Después cree el Jframe, agregué sus respectivas funciones y sus validaciones





Por último, agregué en selección de gestión, su respectiva dirección

```
    private void btnInicioActionPerformed(java.awt.event.ActionEvent evt) {
    VentanaBienvenida ventanabienvenida = new VentanaBienvenida();
    ventanabienvenida.setVisible(true);
    ventanabienvenida.setLocationRelativeTo(null);
    this.setVisible(false);
    }
```

Antes de terminar todo, me fijé y decidí en el eliminar, usar un IF preguntando si realmente quiere eliminar ese objeto de su respectiva tabla, así nos evitamos errores de eliminar cosas o personas que no queríamos

```
int fila = tablaEspecialidades.getSelectedRow();
    if (fila == -1) {
        System.out.println("Debe seleccionar una Especialidad");
        return;
    }

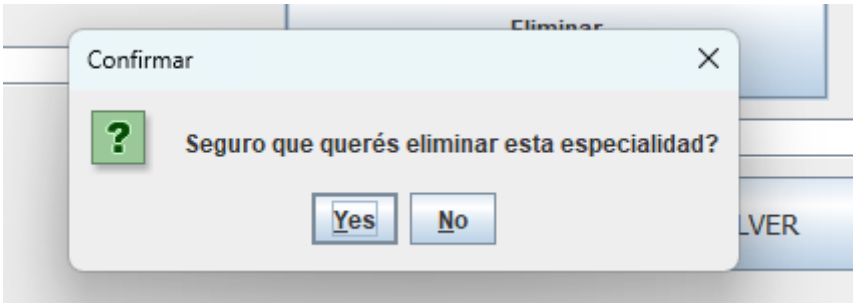
    int id = (int) tablaEspecialidades.getValueAt(fila, 0);

    EspecialidadesDAO dao = new EspecialidadesDAO();

    int confirmar = JOptionPane.showConfirmDialog(null, "Seguro que querés eliminar esta especialidad?", "Confirmar", JOptionPane.YES_NO_OPTION);
    if (confirmar != JOptionPane.YES_OPTION) {
      return;
    }

    dao.eliminar(id);

    JOptionPane.showMessageDialog(null, "Registro eliminado Correctamente");
```



Así en todas las tablas. Una vez hice todo, lo subí al GitHub y terminé con todo este día

2/11/26

Lugar: Laboratorio

Hora: 9:55

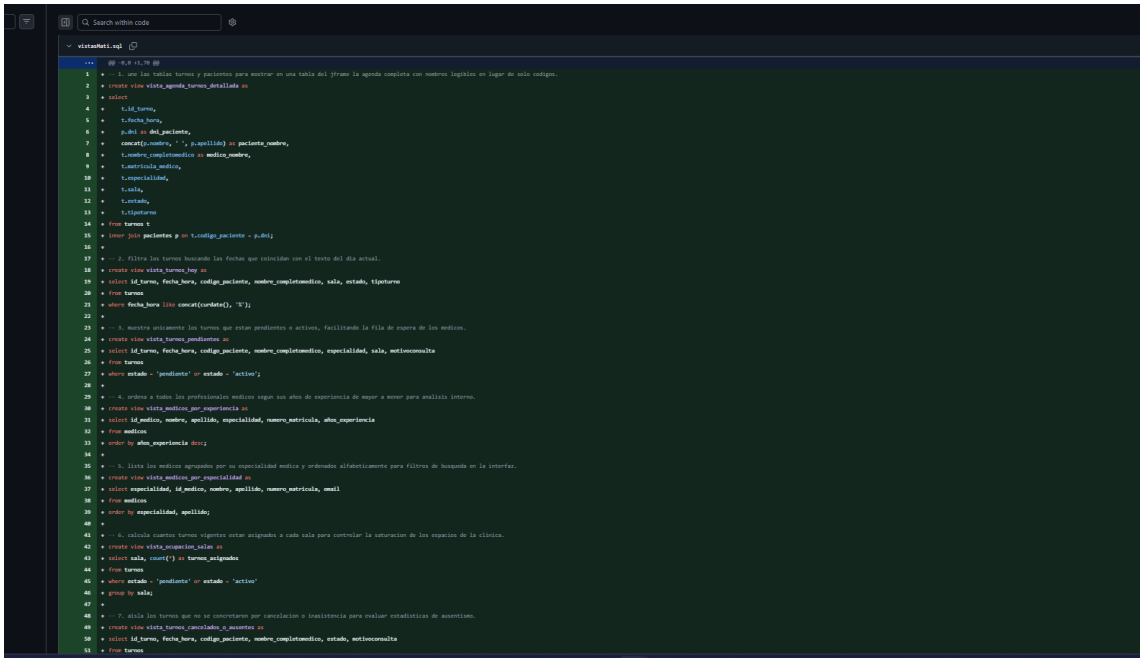
Actividad:

El día hoy me puse a seguir con mis consigas que me quedaban pendientes para esta entrega final



| | | |

Total de Vistas: 10



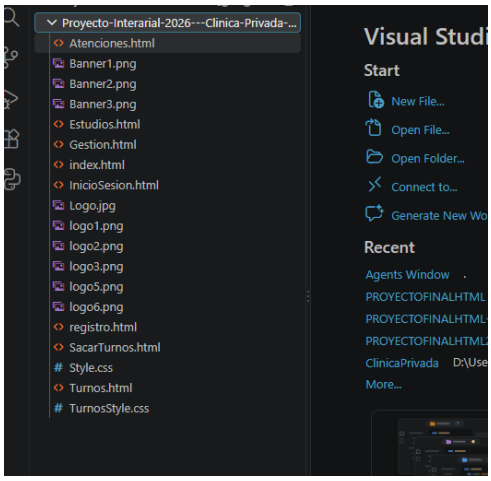
4/11/26

Lugar: Aula Informática

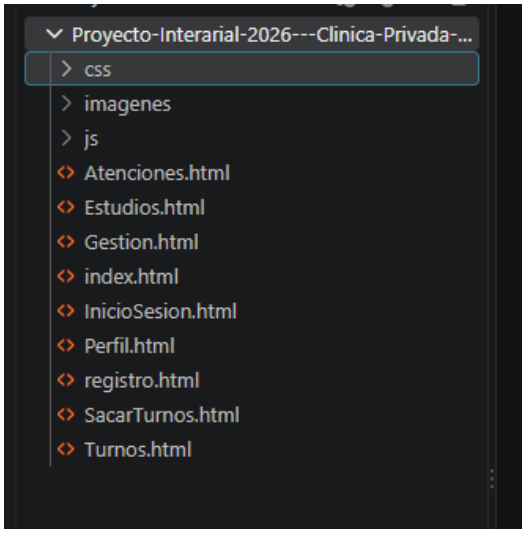
Hora: 7:40

Actividad:

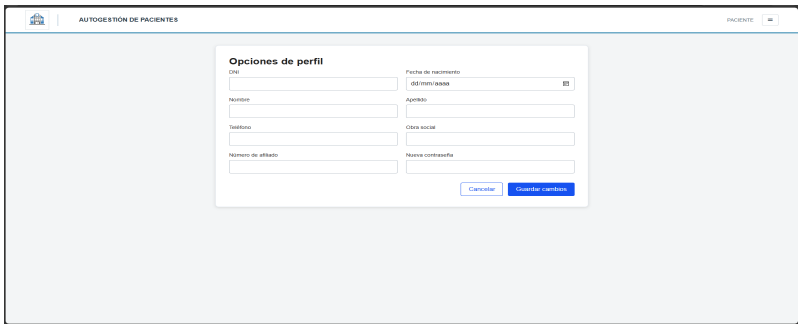
Hoy seguí con la página web. Nos faltá poco para la entrega así que debía terminar la mayoría de cosas. Mi líder me mandó a que terminara el css de la página, ya que se veía feo. Me iba poner a trabajar pero vi que había un serio desorden en la carpeta, así que primero decidí ordenarla. Ordené los js en carpetas distintas, los css, los separé algunos ya que estaban todo metidos en un solo css, además cree su propia carpeta y por último las imágenes también las guarde en una carpeta separado a los html. Pasando de esto



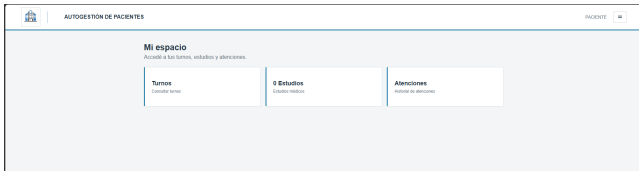
A esto

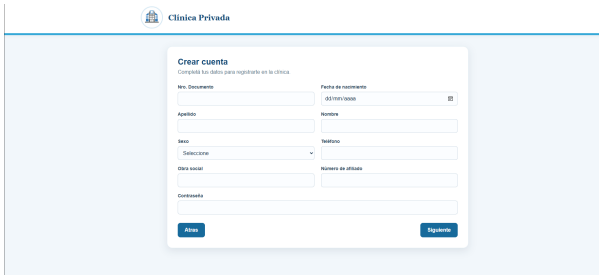


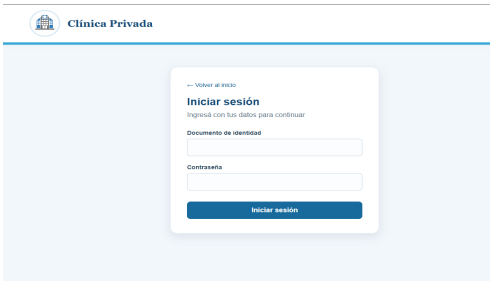
Después de ordenar, continúe con lo que me tocaba, crear una sección para editar los datos del paciente. Armé una pantalla de opciones de perfil, con la misma barra superior que en Gestión, “header” con el logo y un menú desplegable hecho con “button” y una lista “ul, li y a”. El contenido va dentro de un “main” con una “section” que funciona como tarjeta, con un título h1 y un “form”. Los datos se ordenan en un “div” de campos, cada uno con su “label” vinculado por “for” y su input: DNI, fecha de nacimiento (date), nombre, apellido, teléfono “tel”, obra social, número de afiliado y nueva contraseña “contrasena”. Al final puse dos acciones: un enlace a “Cancelar” que vuelve a Gestión y un boton “Guardar cambios”. Los estilos los vinculé con link a paciente.css y perfil.css, y las funciones con script a paciente.js. Finalmente se ve asi



Por último, me puse a hacer el css para decorar toda la página lo que hice fue separé los estilos en cuatro archivos CSS, uno por grupo de pantallas: acceso.css (login y registro), paciente.css (gestión, turnos y menú), perfil.css (perfil) y turno.css (sacar turno). Para el diseño usé una paleta de azules y grises claros, con fondos blancos sobre un fondo gris suave, y la fuente Arial. Con “display: flex” acomodé la barra superior, las filas de campos del registro y los botones, y con “display: grid” armé la grilla de opciones de Gestión y los campos del perfil. A las tarjetas y formularios les di un aspecto más moderno con “border-radius” para las esquinas redondeadas, box-shadow para la sombra y “border” para los bordes. A los campos (input y select) les puse altura, relleno y borde, y con “:focus” resalté el campo que se está usando. A botones y opciones les agregué efectos con “:hover y transition”, y al menú desplegable lo posicioné con position: absolute y lo mostro con la clase .show. Para que todo se adapte a cualquier pantalla usé anchos flexibles con “min()” y “calc()”, finalmente después de realizar todo el css con ayuda de unas clases de tutorial de css que me vi https://www.youtube.com/watch?v=-xtS1QwxfOo&list=PLH_tVOsiVGznLd-dSQP9_ttuxLnSmSWwq&index=1 y aprovechando la IA, pude lograr esto







6/11/26

Lugar: Aula Informática

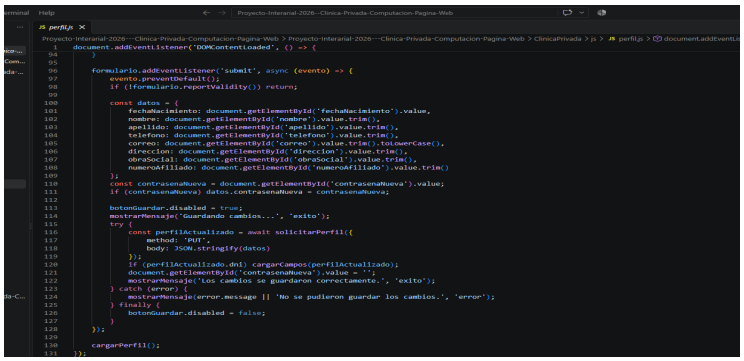
Hora: 7:40

Actividad:

El día de hoy con mi compañero Luciano Villarroel nos pusimos a testear la app de java para que faltaba y si funcionaba todo. Después de testearlo nos dimos cuenta que: Falta agregar un botón de finalizar la app, carteles de turno, cartel de enfermedad del atributo buscar, que confirme que encontró y también un cartel que confirme que edito y aplicar los triggers a java

Antes de eso, me pidieron que prepare la conexión de registrar, iniciar sesión, editar datos para que se pueda conectar a java web y a mysql. Lo que hice fue que use javascript para validar los datos y gestionar las solicitudes. Deje preparado el perfil para comunicarse con una API de java web. Para completar la conexión, el servidor de java web tiene que validar los datos y en base a la librería de conexión “JDBC” los guarda y consulta en MYSQL. Además aplique una validación en DNI, para que no supere un limite de 8 digitos.





```
document.addEventListener('DOMContentLoaded', () => {
    }

    formulario.addEventListener('submit', async (evento) => {
        evento.preventDefault();
        if (!formulario.reportValidity()) return;

        const datos = {
            fechaNacimiento: document.getElementById('fechaNacimiento').value,
            nombre: document.getElementById('nombre').value.trim(),
            apellido: document.getElementById('apellido').value.trim(),
            telefono: document.getElementById('telefono').value.trim(),
            correo: document.getElementById('correo').value.trim().toLowerCase(),
            direccion: document.getElementById('direccion').value.trim(),
            obraSocial: document.getElementById('obraSocial').value.trim(),
            numeroAfiliado: document.getElementById('numeroAfiliado').value.trim()
        };
        const contrasenaNueva = document.getElementById('contrasenaNueva').value;
        if (contrasenaNueva) datos.contrasenaNueva = contrasenaNueva;

        botonGuardar.disabled = true;
        mostrarMensaje('Guardando cambios...', 'exito');
        try {
            const perfilActualizado = await solicitarPerfil({
                method: 'PUT',
                body: JSON.stringify(datos)
            });
            if (perfilActualizado.dni) cargarCampos(perfilActualizado);
            document.getElementById('contrasenaNueva').value = '';
            mostrarMensaje('Los cambios se guardaron correctamente.', 'exito');
        } catch (error) {
            mostrarMensaje(error.message || 'No se pudieron guardar los cambios.', 'error');
        } finally {
            botonGuardar.disabled = false;
        }
    });

    cargarPerfil();
});
```

Una vez termine, lo continúo Villarroel con la conexión a java web y MySQL por suerte, pudimos hacer que funcione todo, finalizando por fin con toda la web.